VIT Chennai · School of Computer Science and Engineering (SCOPE)

BACSE202 · Database Systems · Slot D2+TD2 · Fall 2026-27

Digital Assignment 3

Modern Database Technology: Neo4j Graph Database

Campus Connect

A social and academic platform for the SCOPE community, extended with a graph database

Item	Detail
Course	Database Systems
Course code	BACSE202
Slot	D2+TD2
Project	Campus Connect (DA1 relational design, DA2 SQL and PL/SQL work, DA3 graph database)
Selected technology	Neo4j 2026.10.0 (enterprise), queried with Cypher
Main database	SQLite (system of record), with Neo4j added alongside it
Submission	Digital report and project demonstration

Submitted by

Name	Registration number
Aayush Bhardwaj	25BCE1315
Anindya Ghosh	25BCE1326
T Abilash	25BCE1285

Faculty: Dr. Bhuvaneswari A

GitHub: https://github.com/aayushbhardwaj2506/CAMPUS-CONNECT/tree/da3-neo4j-graph

Abstract

Campus Connect is a university social and academic platform built for students and faculty of the School of Computer Science and Engineering (SCOPE), VIT Chennai. It combines academic groups, a campus feed, messaging, clubs, communities, events, projects, peer help and slot-based class registration in one application. The application stores all of its everyday data in a relational SQLite database designed in DA1 and extended in DA2.

For Digital Assignment 3 the project is extended with Neo4j, a graph database. The reason is that the most interesting questions on a campus network are questions about relationships: who shares classes and clubs with whom, how two students are linked through other people, and which clubs or teammates to suggest. These are multi-hop questions that need long chains of joins in SQL but are short pattern matches in Cypher.

This report describes the graph model, the implementation in Neo4j, the import of the full Campus Connect dataset (1,324 nodes and 7,482 relationships), the core create, read, update and delete operations, and ten technology-specific queries, each shown with its Cypher and its result in table and graph form. It also explains how Neo4j is used inside the running application, and compares it with the relational design to justify why a graph database is suitable for this application.

Contents

1. Introduction 3
2. Overview of the Campus Connect Application 4
3. Why Campus Connect Needs a Graph Database 9
4. Graph Model (Schema Design) 10
5. Implementation in Neo4j 15
6. Creation and Import of the Dataset 19
7. Core Operations (Create, Read, Update, Delete) 25
8. Technology-Specific Queries 29
9. Neo4j in the Application 41
10. Project Demonstration 47
11. Comparison: Why Neo4j Is Suitable for Campus Connect 48
12. Conclusion 49

1. Introduction

1.1 Background

Campus Connect was built across the three digital assignments of the course. DA1 produced the relational design of the platform (36 relations with their keys, constraints and assumptions). DA2 extended that design with SQL and PL/SQL work. The running application uses an SQLite database built from the same design, and it is the project that DA3 extends.

1.2 Objective of DA3

The assignment asks for the same mini-project to be extended with one modern database technology other than MongoDB. We selected a graph database, Neo4j, because the central idea of Campus Connect is connection: people are linked to classes, groups, clubs, communities and projects, and through those to each other.

1.3 How this report meets the DA3 requirements

DA3 requirement	Where it is covered
Database schema / model design	Section 4: graph model, schema diagram, example with real data, node and relationship tables, derived CONNECTED relationship, design decisions, constraints
Implementation using the selected technology	Section 5: Neo4j instance, driver connection, constraints, labels and relationship types
Creation / import of sample datasets	Section 6: dataset, import script, import run and the resulting graph
Core operations and technology-specific queries	Section 7 (create, read, update, delete) and Section 8 (ten Cypher queries with results)
Brief comparison: why the database is suitable	Section 11: comparison of Neo4j with the relational design, and suitability
Digital report and project demonstration	This report; Section 9 (Neo4j in the application) and Section 10 (demonstration)

1.4 Structure of the report

Because DA3 builds on the full project, Section 2 first gives an overview of the complete Campus Connect application and its relational design, so that the role of the graph database is clear. Section 3 explains why a graph database is needed. Sections 4 to 8 then concentrate on Neo4j: model, implementation, dataset, operations and queries. Sections 9 and 10 show Neo4j working inside the application, and Sections 11 and 12 compare the two databases and conclude.

2. Overview of the Campus Connect Application

2.1 Purpose and users

Campus life is usually spread across portals, chat groups, notice boards and spreadsheets. Campus Connect brings these into one place. When a student account is created, the system uses the student's academic identity (school, department, programme, batch year, current semester and registered classes) to place the student automatically in the right groups. From there the student can follow announcements, post to the campus feed, chat, join clubs and communities, collaborate on projects and ask for peer help.

There are three kinds of account: students, faculty, and staff (administration and moderation). Faculty can publish announcements and study material, and staff handle moderation reports.

2.2 Technology stack

Layer	Technology
Server	Node.js with the Express 4 web framework, express-session for logins, bcryptjs for password hashing
Pages	EJS server-side templates, CSS and plain JavaScript; a dark pixel-art visual theme
Main database	SQLite, accessed through Node's built-in synchronous driver (node:sqlite). The database is one file, campus_connect.db
Graph database (DA3)	Neo4j 2026.10.0 enterprise, run locally in Neo4j Desktop, accessed with the official neo4j-driver 6
Deployment options	Run locally with npm start; render.yaml for a Render web service; optional Docker files

2.3 System architecture

The browser talks to an Express server. Every everyday feature reads and writes SQLite, which is the system of record. Neo4j holds a copy of the relationship data. An import module copies data from SQLite into Neo4j, and a Sync graph button on the Campus graph page repeats the copy on demand. Only the Campus graph page asks Neo4j questions; the rest of the application does not depend on it.

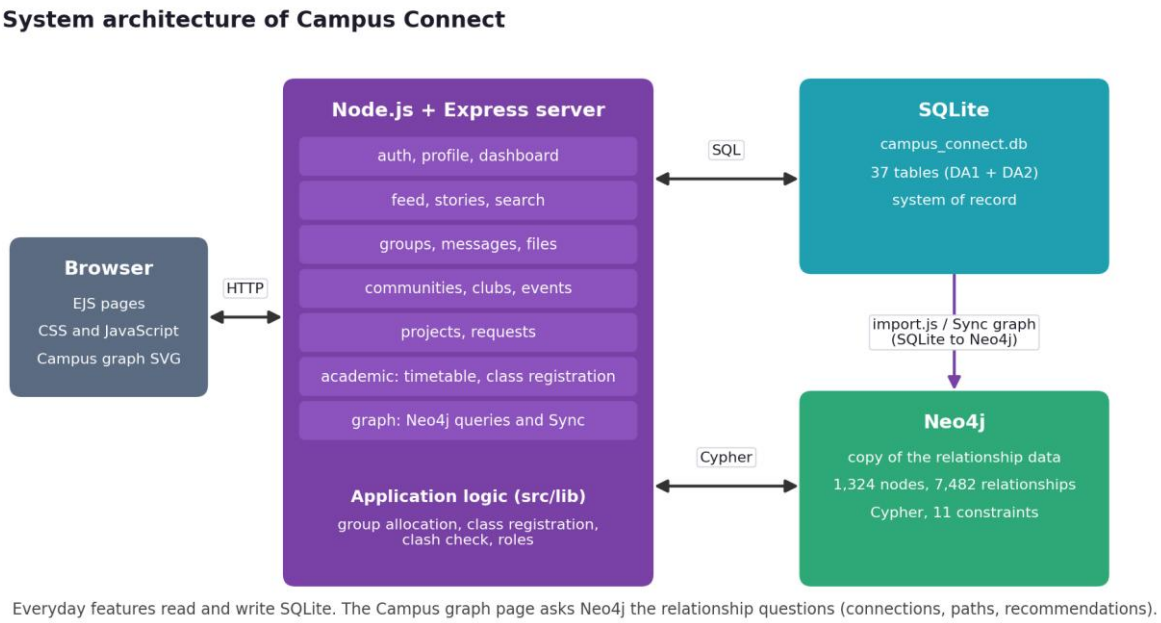


Figure 1. System architecture of Campus Connect: SQLite is the system of record, Neo4j is a graph copy used for relationship questions.

2.4 Relational design in SQLite

The relational design contains 37 tables: the 36 relations of the DA1 design plus one table added in DA2. DA2 also added a few nullable columns, each tagged in the schema file with the reason for the change. Foreign keys, unique constraints and check constraints carry the integrity rules.

Module	Tables
Academic structure (8)	School, Department, Program, Course, Building, Room, Class, Timetable
People (4)	User, Profile, Student, Faculty
Groups and notices (4)	Group, Group_Membership, Announcement, Study_Material
Messaging (3)	Conversation, Conversation_Participant, Message
Social feed (4)	Post, Comment, Reaction, Story
Community life (5)	Community, Community_Membership, Club, Club_Membership, Event
Projects (3)	Project, Project_Team, Project_Membership
Requests (2)	Request_Category, Request
Platform (4)	File, Notification, Report, Audit_Log

2.5 Application features

The following features are implemented and were used to generate the data that Neo4j stores. The screenshots in this section are taken from the running application.

Feature	What it does
Accounts and profiles	Sign-in with role selection; profile with bio, birthday, programme, registration number, batch and semester

Feature	What it does
Automatic group allocation	On account creation the student joins the all-students, school, department, programme, batch and current-semester course groups; faculty join their department groups
Class registration	A student picks a slot, then a course, then a faculty member. The class group and timetable are built from the choice, and slot clashes are blocked
Campus feed and stories	Posts with visibility (campus, public, private), comments, reactions, and 24-hour stories
Messaging	Direct and group conversations, including a chat for each course group
Communities, clubs and events	Communities and clubs with categories and member roles; events hosted by communities
Projects	Projects with teams, roles and join requests
Requests and peer help	Requests with categories, priority and status, answered by other students
Academic directory	Schools, departments, programmes, courses, buildings and rooms, timetables and study material
Notifications and moderation	Notification centre, reports, and an audit log
Campus graph (DA3)	Neo4j-powered view of connections, recommendations and shortest paths

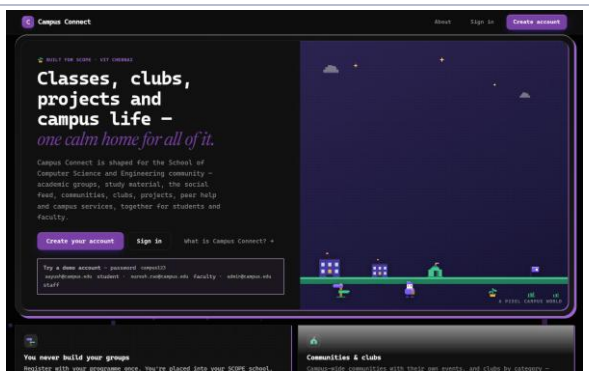


Figure 2. Landing page of Campus Connect.

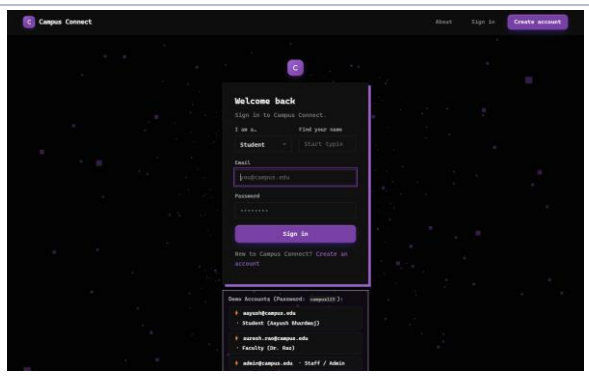


Figure 3. Sign-in page with role and name pickers.

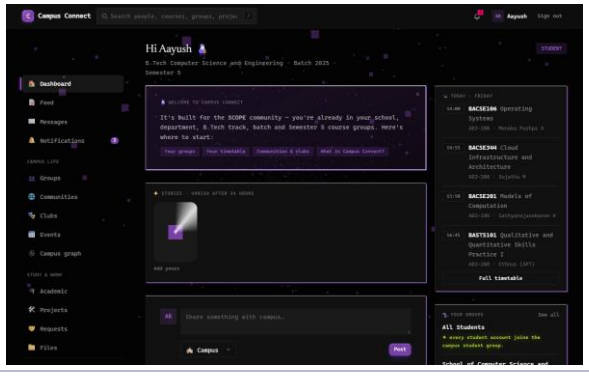


Figure 4. Dashboard: today's timetable, stories, groups.

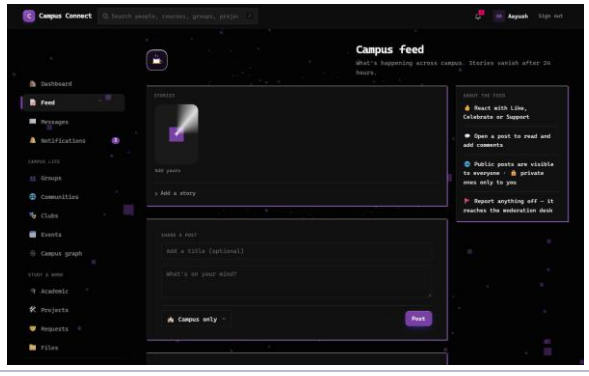


Figure 5. Campus feed with posts and stories.

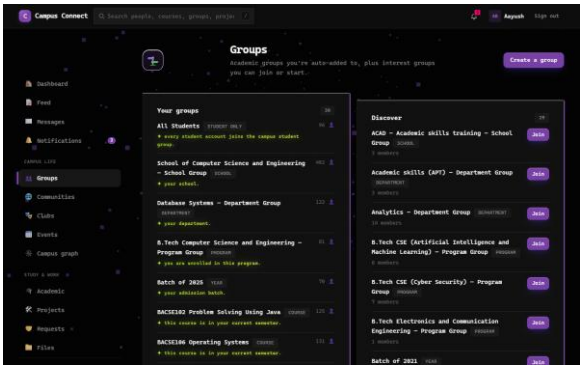


Figure 6. Groups: auto-allocated groups and groups to discover.

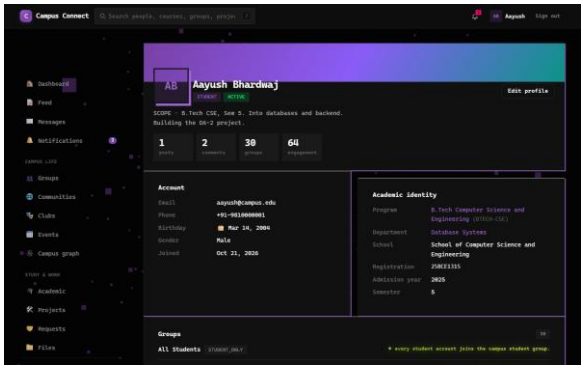


Figure 7. Profile with academic identity and groups.

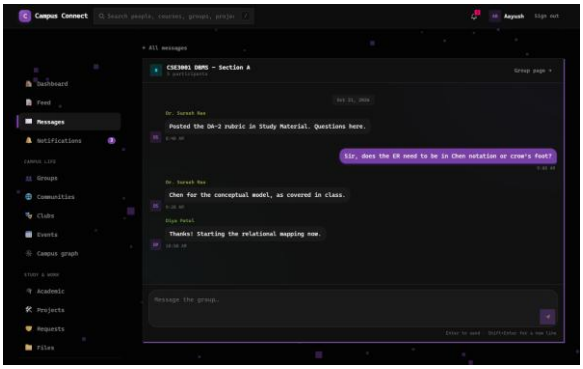


Figure 8. Group chat of a course group.

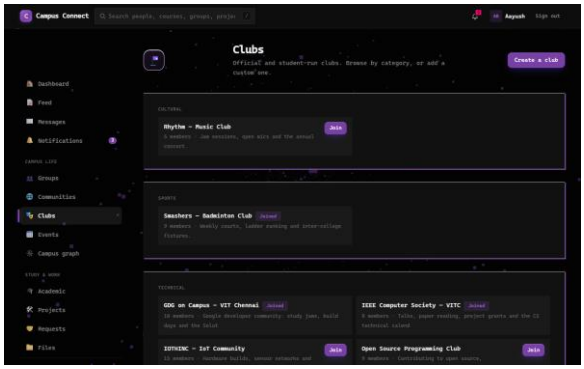


Figure 9. Clubs by category.

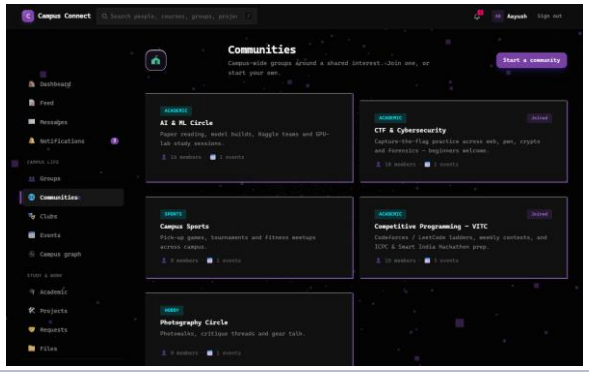


Figure 10. Communities.

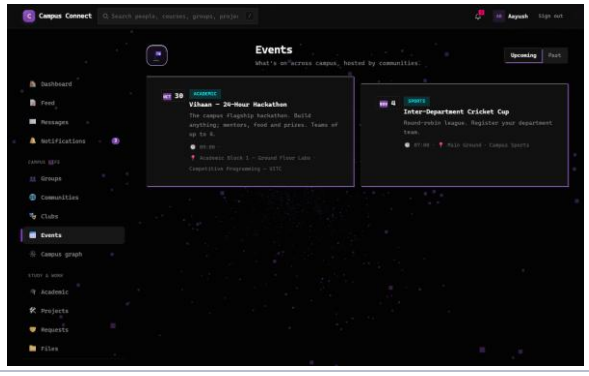


Figure 11. Events hosted by communities.

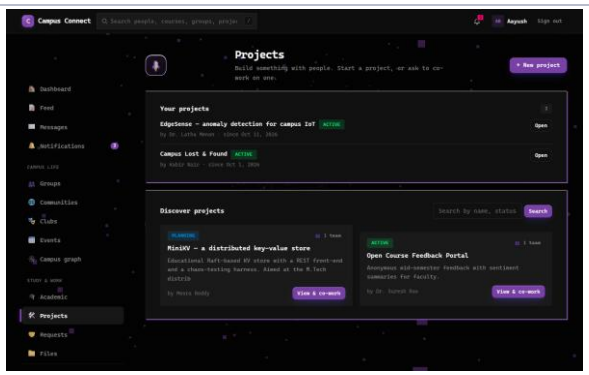


Figure 12. Projects and teams.

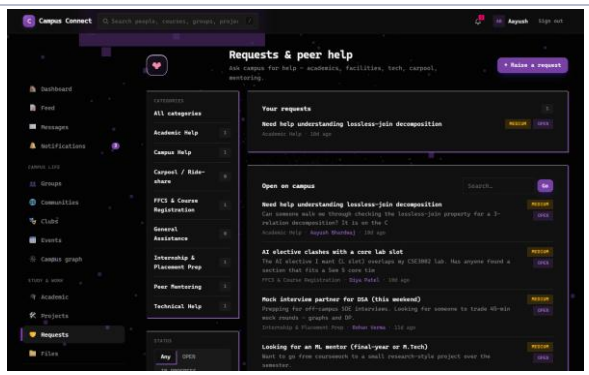


Figure 13. Requests and peer help.

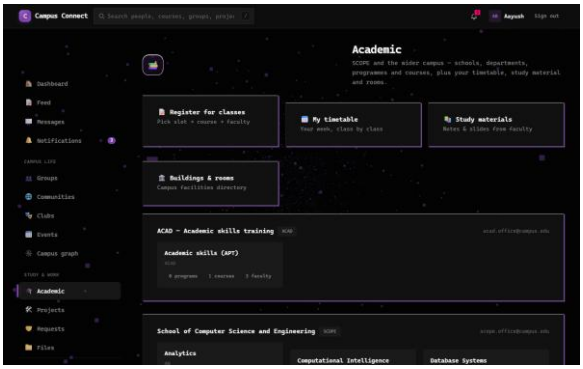


Figure 14. Academic directory.

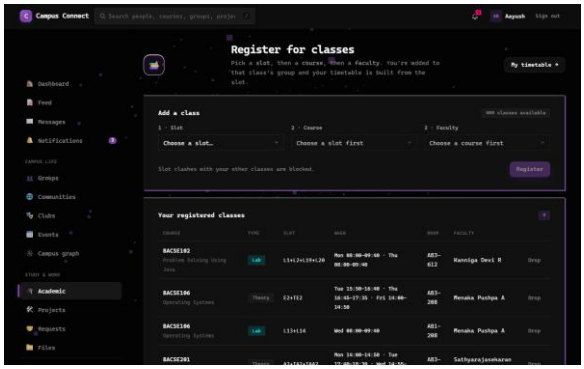


Figure 15. Class registration (slot, course, faculty) and registered classes.

3. Why Campus Connect Needs a Graph Database

Most of what makes a campus network useful is not a single record but the links between records. A student is in a club, the club has other members, those members take the same courses, and some of them work on a project together. A relational database can store all of this, but every additional hop becomes another join, and the query grows harder to write and slower to run.

In a graph database a relationship is stored as a first-class object next to the data. Following it does not search a table; it follows a direct reference from one node to the next. The cost of a query depends on how much of the graph is touched and not on the total size of the tables.

3.1 Questions the application needs to answer

- Who shares the most with me (classes, clubs, communities, projects)?
- Who are the people I may know, that is, friends of my connections?
- Which clubs do my connections belong to that I have not joined?
- How many introductions separate two students, and who would introduce me?
- Who are the best-connected people on campus?
- Which two clubs share the most members?

Section 8 answers each of these with a Cypher query. Section 11 compares them with the equivalent SQL.

4. Graph Model (Schema Design)

Neo4j does not need a fixed schema, but a model is still needed to decide what becomes a node, what becomes a relationship and what becomes a property. We designed the model from the relational schema using three rules.

1. **An entity becomes a node.** School, Department, Program, Course, Class, Person (with the extra labels Student and Faculty), Club, Community, Event, Project and Group each become a node label.
2. **A foreign key or a junction table becomes a relationship.** Club_Membership, Community_Membership, Project_Membership and Group_Membership are many-to-many links. In the graph each is just an edge between two nodes.
3. **Extra columns of a junction table become relationship properties.** For example, the role of a member is stored on the MEMBER_OF relationship.

4.1 The graph schema

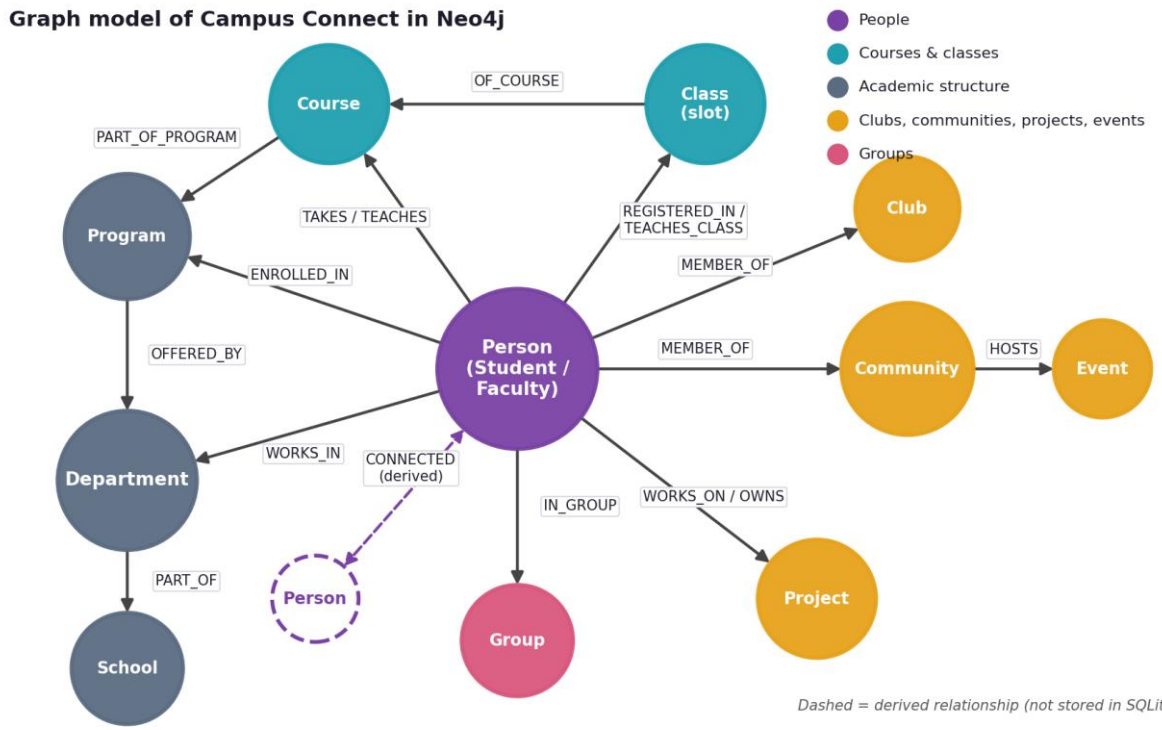


Figure 16. Graph model of Campus Connect. Circles are node labels and arrows are relationship types. The dashed arrow is a derived relationship.

4.2 From tables to a graph: an example

The idea behind the whole model is shown by one junction table. In SQLite, club membership is stored in the Club_Membership table, one row per membership, with the role in a column and the person and the club referenced by foreign keys. In Neo4j the same row becomes one MEMBER_OF relationship between a Person node and a Club node, and the role becomes a property of that relationship. The figure uses real rows from the Campus Connect database.

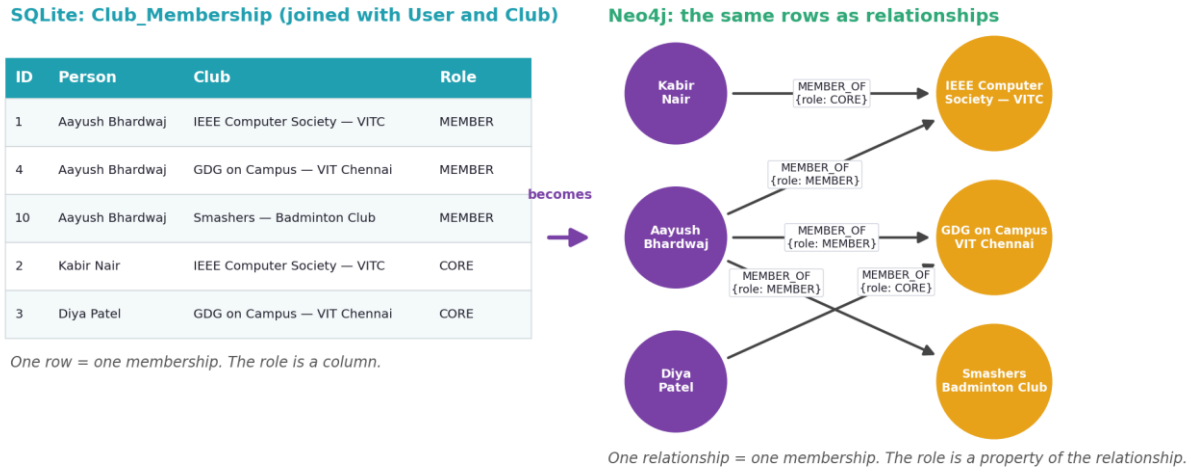


Figure 17. A junction table of the relational design (left) and the same rows as relationships in the graph (right).

4.3 The model filled with real data

The schema diagram above shows types. The next figure shows one real student, the demonstration account, and some of the nodes and relationships that surround that student in the database: the programme and department, a registered class with its course and teacher, a club, a community, a group, and a connected student. Each circle is a node, each arrow a relationship of one of the types in the schema.

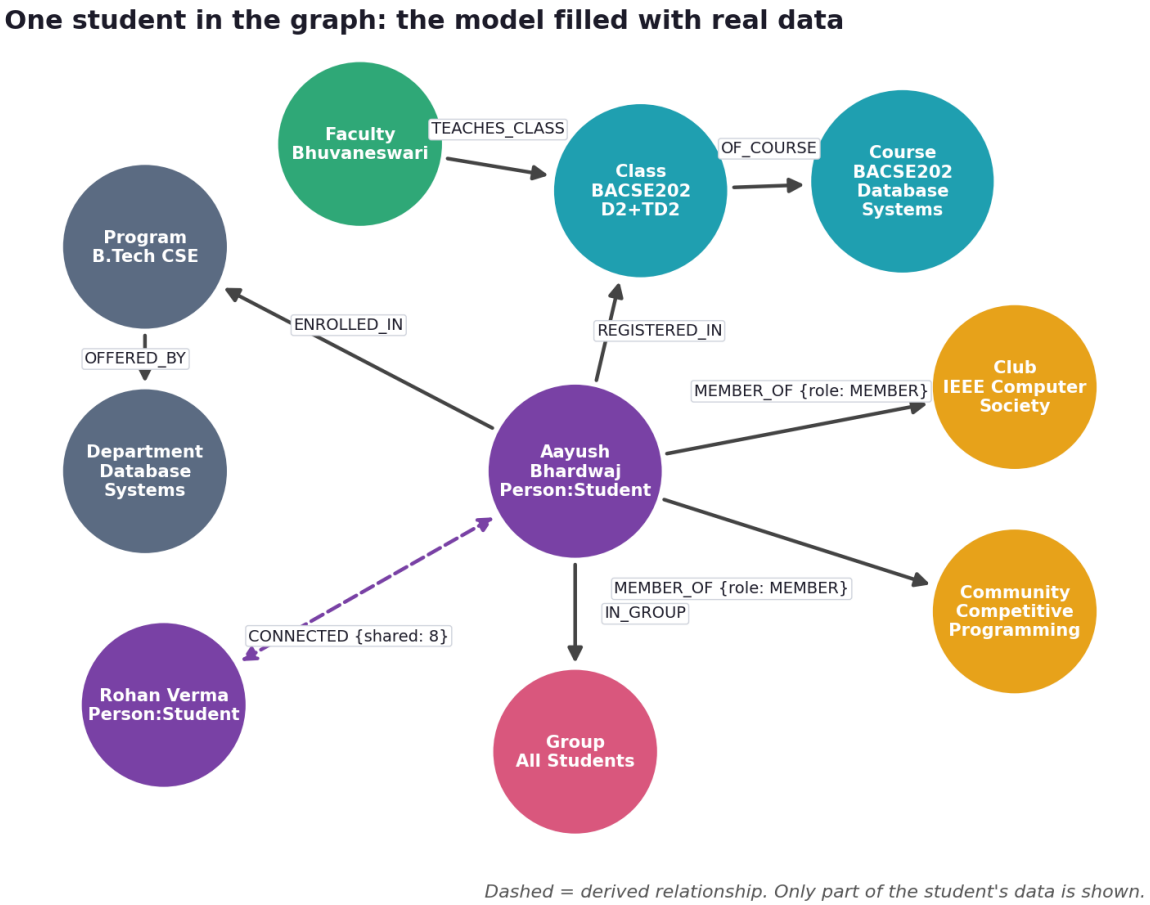


Figure 18. One student in the graph: the model filled with real data (part of the student's neighbourhood).

4.4 Nodes and their properties

Node label	Key properties	Created from (SQLite)	Count
`Person:Student`	userId, name, email, regNo, batch, semester	User + Student	96
`Person:Faculty`	userId, name, email, designation, specialization	User + Faculty	312
`Class`	classId, number, slot, name	Class (slot-registered)	400
`Course`	courseId, code, title, credits, semester	Course	20
`Program`	programId, name, code, level	Program	8
`Department`	deptId, name, code	Department	9
`School`	schoolId, name, code	School	5
`Club`	clubId, name, category	Club	8
`Community`	communityId, name, type	Community	5
`Event`	eventId, name, date	Event	7
`Project`	projectId, name, status	Project	4
`Group`	groupId, name, type	Group	450

4.5 Relationships and their sources

Relationship	From to	Comes from (SQLite)	Count
`ENROLLED_IN`	Student to Program	Student.Program_ID	96
`WORKS_IN`	Faculty to Department	Faculty.Department_ID	312
`TAKES`	Student to Course	Derived: programme and current semester	117
`TEACHES`	Faculty to Course	Class.Faculty_ID, Class.Course_ID	142
`TEACHES_CLASS`	Faculty to Class	Class.Faculty_ID	400
`REGISTERED_IN`	Student to Class	Class-group membership (slot, course, faculty registration)	757
`OF_COURSE`	Class to Course	Class.Course_ID	400
`MEMBER_OF {role}`	Person to Club / Community	Club_Membership, Community_Membership	130
`WORKS_ON {role}`	Person to Project	Project_Membership, Project_Team	17
`OWNS`	Person to Project	Project.User_ID	4
`IN_GROUP`	Person to Group	Group_Membership	3,162
`HOSTS`	Community to Event	Event.Community_ID	7
`OFFERED_BY`	Program to Department	Program.Department_ID	8
`OWNED_BY`	Course to Department	Course.Department_ID	20
`PART_OF_PROGRAM`	Course to Program	Course.Program_ID	20
`PART_OF`	Department to School	Department.School_ID	9
`CONNECTED {shared, via}`	Person to Person	Derived inside Neo4j (see 4.6)	1,881

4.6 The derived CONNECTED relationship

SQLite has no table that says two people are connected. We create this relationship inside Neo4j, right after the import. Two people are CONNECTED when they share at least two items out of the clubs, communities, projects, courses and registered classes they belong to. The number of shared items is stored on the relationship as shared, and their names as via.

The threshold of two is deliberate. With a threshold of one, almost everyone shares something with everyone, every path is one hop long and the graph says nothing. With a threshold of two, paths of one to three hops appear, which is what makes shortest-path and recommendation queries meaningful.

4.7 Design decisions

Several choices in the model were made deliberately. Each is listed with its reason.

Decision	Reason
One Person node per account, with the extra labels Student and Faculty	Students and faculty are both users with the same account data. A shared label lets one query cover everyone, and the extra labels allow queries on students only or faculty only.
Class is a node of its own (course, slot and faculty section)	A registration links a student to a specific section, not just to a course. With a Class node we can ask who shares a class with whom and who teaches it.
Junction tables become relationships	A membership is a link between two things, not an entity with a life of its own. The link is stored directly, and traversing it needs no join.
Roles and reasons are relationship properties	The role is stored on MEMBER_OF and WORKS_ON, and the reason two people are connected is stored on CONNECTED (shared, via), so results can be explained.
CONNECTED is derived, not copied	SQLite has no table for it. It is computed after the import, so the rule (at least two shared items) can be changed and the graph rebuilt.
IN_GROUP is stored but not used to derive CONNECTED	The all-students and school groups contain nearly everyone, so using them would connect everybody. Only clubs, communities, projects, courses and classes express a real shared interest.
Relationship direction follows the meaning (Person to Club, Faculty to Class)	The direction documents the meaning of the link. Neo4j can traverse a relationship in either direction, so queries are not restricted by it.
Only relationship data is copied to Neo4j	Posts, messages and similar tabular data answer no relationship question. SQLite stays the system of record, and the graph stays small and focused.
A uniqueness constraint on every node id	It prevents duplicate nodes and gives an index, so a node can be found quickly by its id.

4.8 Constraints

Each node label has a uniqueness constraint on its identifier, which also creates an index. There are 11 constraints, so a node can be found by its id quickly and duplicates are rejected.

Label	Unique property	Label	Unique property
Class	classId	Club	clubId
Community	communityId	Course	courseId
Department	deptId	Event	eventId
Group	groupId	Person	userId
Program	programId	Project	projectId
School	schoolId		

5. Implementation in Neo4j

This section shows how Neo4j was set up and connected to the application, and how the model of Section 4 was created in the database.

5.1 Neo4j environment

Neo4j runs as a local instance named campus, created in Neo4j Desktop 2.3. The server version is 2026.10.0 (enterprise edition) and the instance listens for Bolt connections on port 7687. Figure 19 shows the instance running.

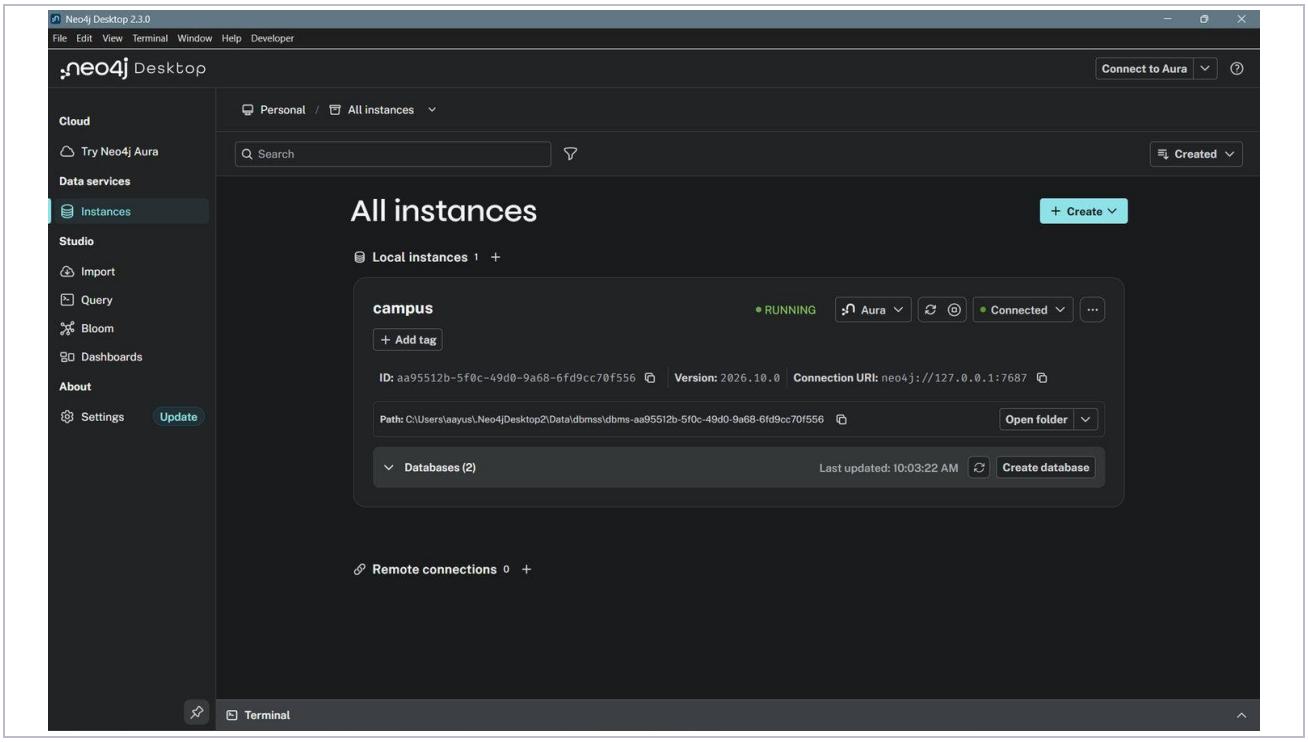


Figure 19. Neo4j Desktop: the campus instance is running (version 2026.10.0).

5.2 Connecting the application

The application connects through the official neo4j-driver package. A small module, src/graph/neo4j.js, creates one shared driver, reads the connection settings (address, user and password) from a git-ignored environment file, runs read and write queries with timeouts, and converts Neo4j integers into plain JavaScript numbers. Keeping all Neo4j access in this one module means the rest of the application never handles driver details.

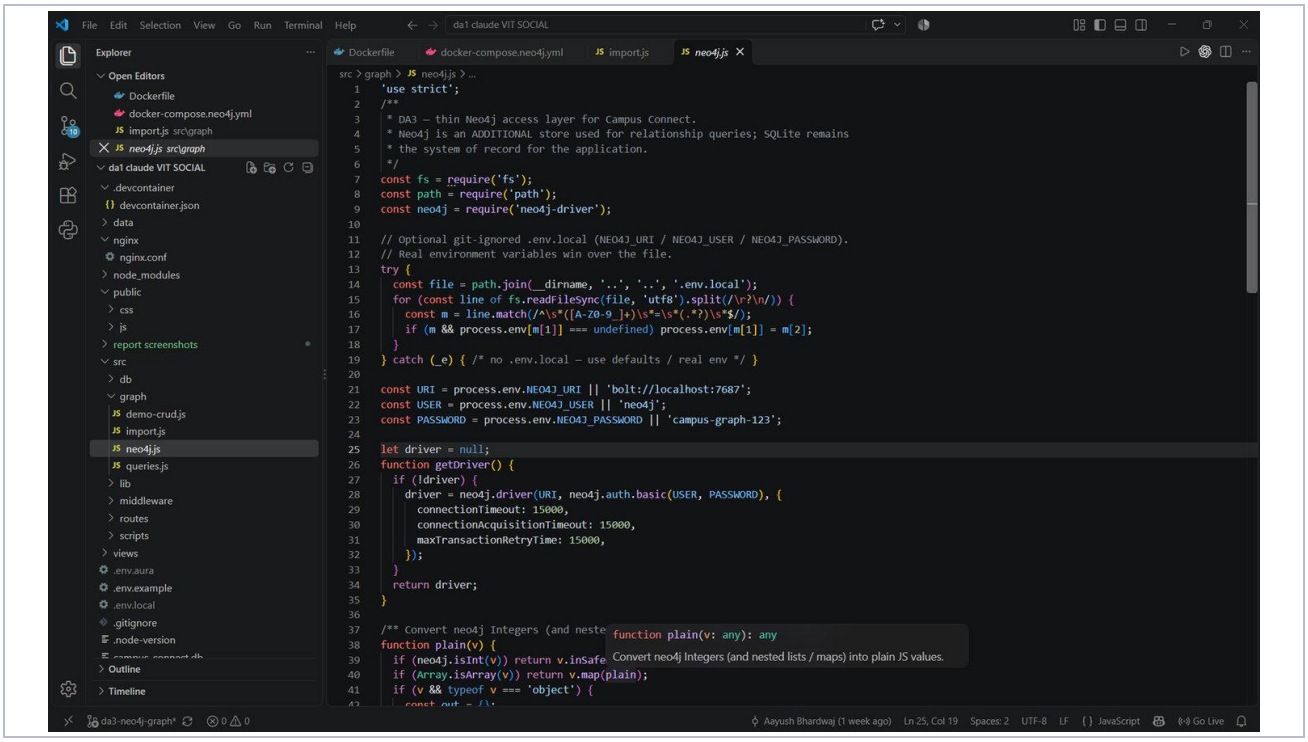


Figure 20. src/graph/neo4j.js: the driver wrapper that connects the application to Neo4j.

5.3 Creating the constraints

The constraints are created by the import script before any data is loaded, so that duplicates can never enter the graph. The script uses CREATE CONSTRAINT … IF NOT EXISTS, so it can be run again safely. The result can be inspected in Neo4j Browser with SHOW CONSTRAINTS.

```
SHOW CONSTRAINTS YIELD name, labelsOrTypes, properties
RETURN labelsOrTypes[0] AS label, properties[0] AS property ORDER BY label
```

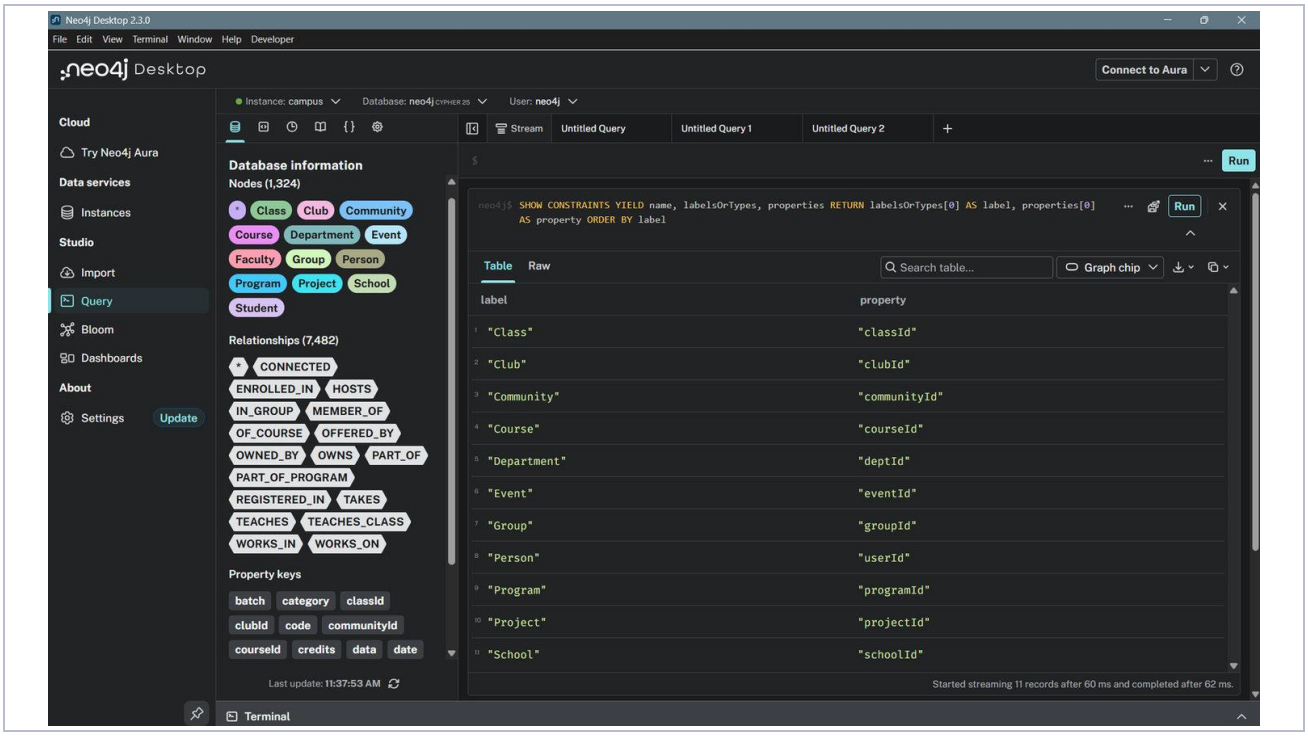


Figure 21. The 11 uniqueness constraints, one for each node label.

5.4 Verifying the model in the database

The node labels and relationship types that exist in the database can be listed with two built-in procedures. The lists match the model of Section 4: eleven node labels (plus Student and Faculty as extra labels on Person) and seventeen relationship types.

```
CALL db.labels()
CALL db.relationshipTypes()
```

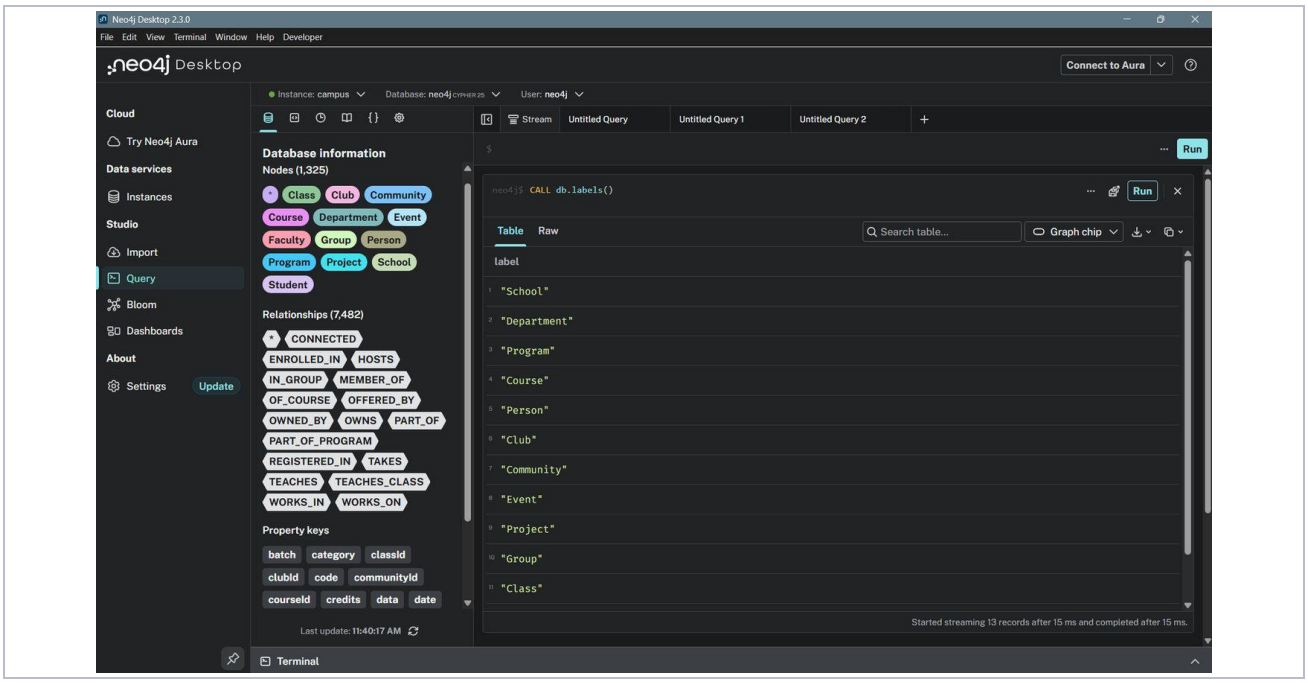


Figure 22(a). Database structure: node labels in the database.

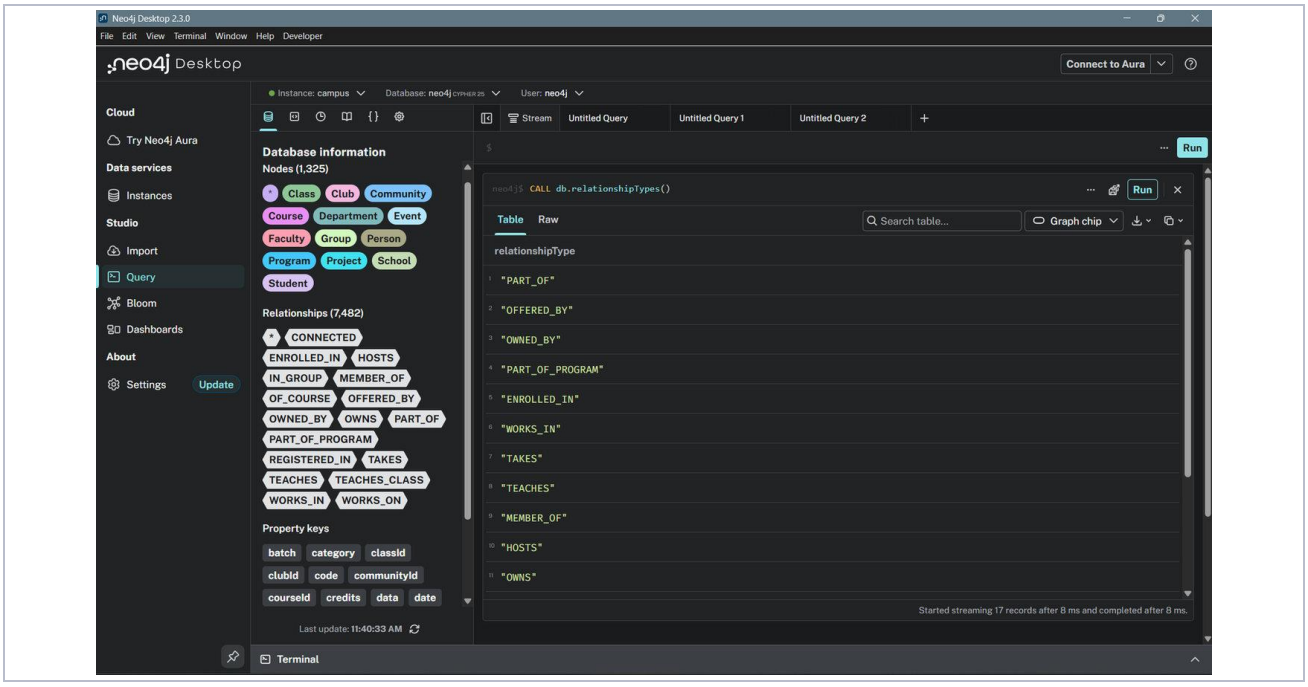


Figure 22(b). Database structure: relationship types in the database.

5.5 Project files for the graph

File	Purpose
`src/graph/neo4j.js`	Driver wrapper: connection settings, read and write helpers, integer conversion
`src/graph/import.js`	Creates constraints, copies nodes and relationships from SQLite, derives CONNECTED
`src/graph/queries.js`	The Cypher queries used by the application and in this report
`src/graph/demo-crud.js`	Terminal demonstration of create, read, update and delete
`src/routes/graph.routes.js`	The /graph page and the Sync graph action
`views/graph/index.ejs`	The Campus graph page: neighbourhood diagram, counts, query results with their Cypher

6. Creation and Import of the Dataset

6.1 The dataset

The dataset is the full Campus Connect database. It contains 409 user accounts (96 students and 312 faculty), the Fall 2026-27 class catalogue of 400 slot-based classes with their faculty, rooms and timetable rows, the academic structure above them, the automatically allocated groups, club, community and project memberships, and each student's class registrations. It was created by the seed script from the class list, the course allocation report and demonstration accounts, and it includes demonstration student accounts generated by the seed.

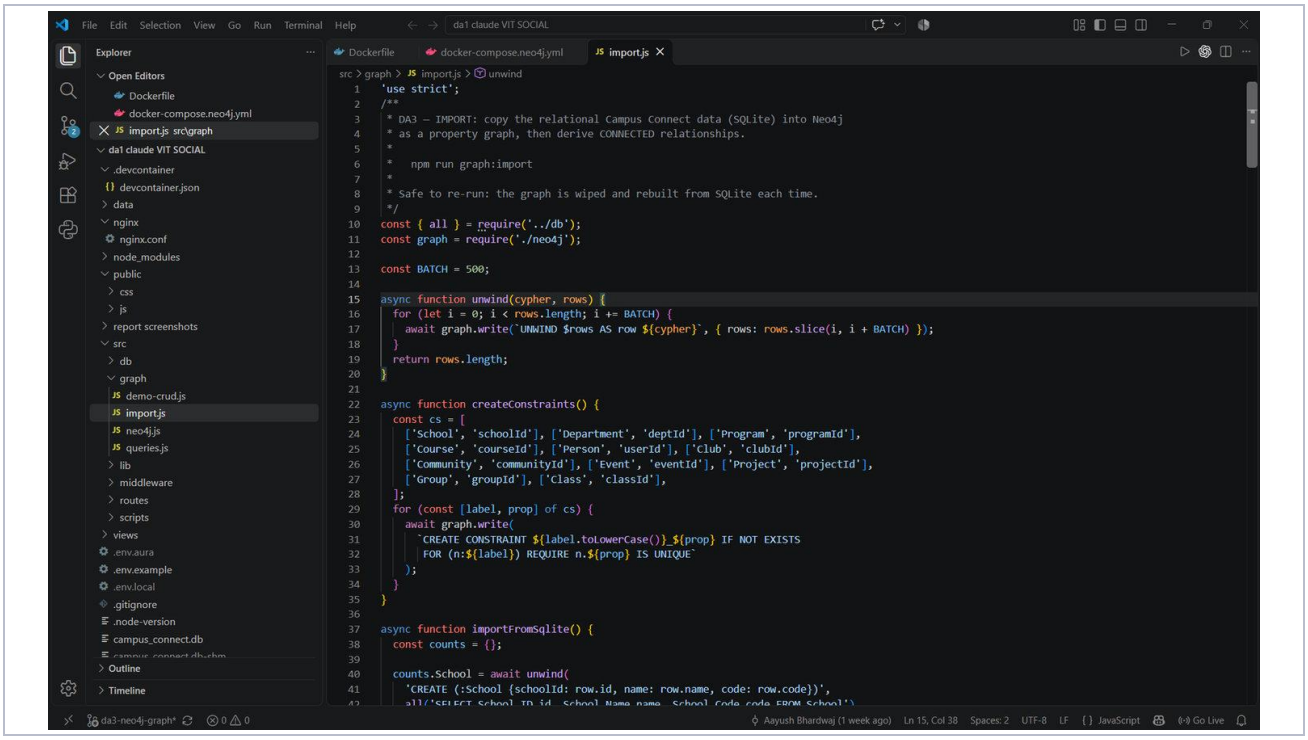
The graph is built from this data by import. It is not typed in by hand, so the graph and the application always describe the same people and classes.

6.2 The import script

The import is a Node.js script, src/graph/import.js, run with npm run graph:import. It reads the SQLite tables with ordinary SQL and writes the graph with Cypher, in batches of 500 rows using UNWIND. The script performs four steps:

1. Wipe the existing graph, so the import can be repeated.
2. Create the 11 uniqueness constraints.
3. Create the nodes and, with each node, the relationships that come from its foreign keys (for example a Student node and its ENROLLED_IN relationship).
4. Derive the CONNECTED relationships between people who share at least two items.

The first screenshot below shows the start of the script: the batching helper and the constraint list. The second shows importFromSqlite(), which reads the School, Department, Program, Course and Student tables and creates the nodes and their relationships; classes, clubs, communities, projects, groups and faculty are imported the same way in the rest of the function.



```
'use strict';
/**
 * DA3 — IMPORT: copy the relational Campus Connect data (SQLite) into Neo4j
 * as a property graph, then derive CONNECTED relationships.
 *
 *   npm run graph:import
 *
 * Safe to re-run: the graph is wiped and rebuilt from SQLite each time.
 */
const { all } = require('../db');
const graph = require('./neo4j');

const BATCH = 500;

async function unwind(cypher, rows) {
  for (let i = 0; i < rows.length; i += BATCH) {
    await graph.write(`UNWIND $rows AS row ${cypher}`, { rows: rows.slice(i, i + BATCH) });
  }
  return rows.length;
}

async function createConstraints() {
  const cs = [
    ['School', 'schoolId'], ['Department', 'deptId'], ['Program', 'programId'],
    ['Course', 'courseId'], ['Person', 'userId'], ['Club', 'clubId'],
    ['Community', 'communityId'], ['Event', 'eventId'], ['Project', 'projectId'],
    ['Group', 'groupId'], ['Class', 'classId'],
  ];
  for (const [label, prop] of cs) {
    await graph.write(
      `CREATE CONSTRAINT ${label.toLowerCase()}_${prop} IF NOT EXISTS
       FOR (n:${label}) REQUIRE n.${prop} IS UNIQUE`
    );
  }
}

async function importFromSqlite() {
  const counts = {};

  counts.School = await unwind(
    'CREATE (:School {schoolId: row.id, name: row.name, code: row.code})',
    all('SELECT School_ID id, School_Name name, School_Code code FROM School')
  );

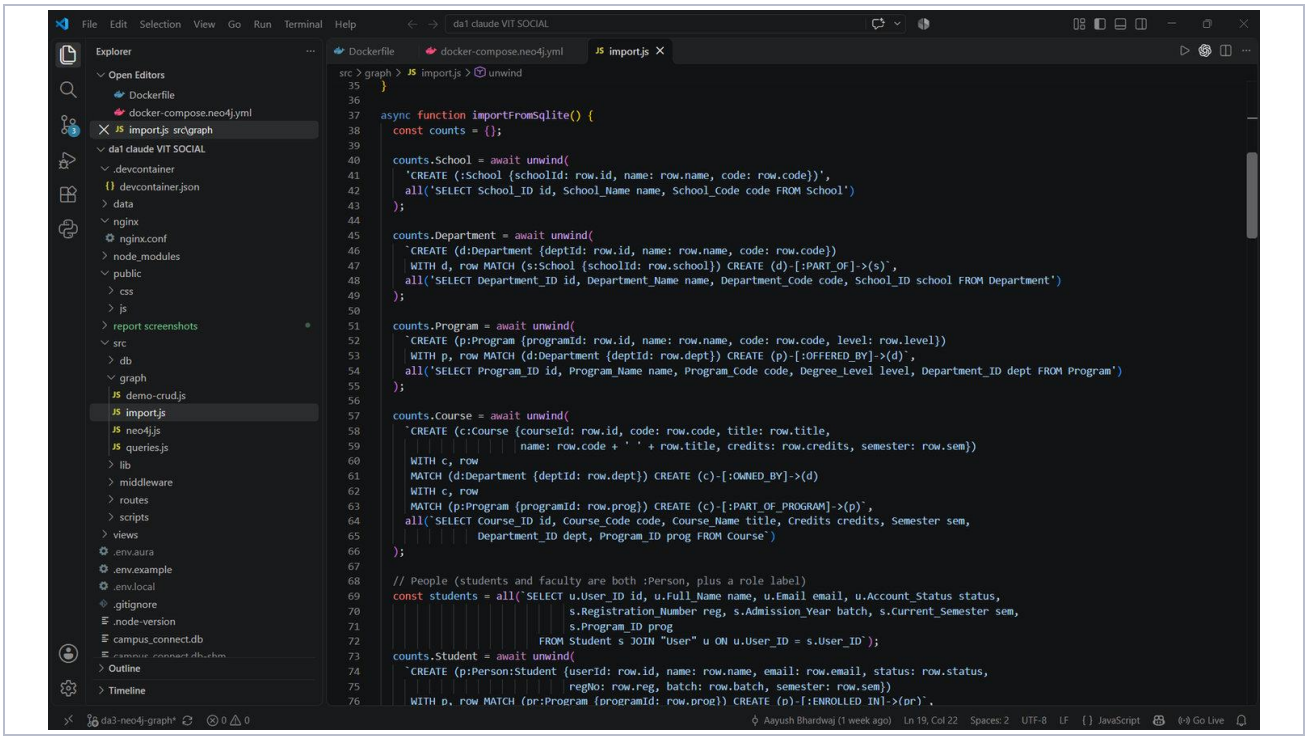
  counts.Department = await unwind(
    `CREATE (d:Department {deptId: row.id, name: row.name, code: row.code})
     WITH d, row MATCH (s:School {schoolId: row.school}) CREATE (d)-[:PART_OF]->(s)`,
    all('SELECT Department_ID id, Department_Name name, Department_Code code, School_ID school FROM Department')
  );

  counts.Program = await unwind(
    `CREATE (p:Program {programId: row.id, name: row.name, code: row.code, level: row.level})
     WITH p, row MATCH (d:Department {deptId: row.dept}) CREATE (p)-[:OFFERED_BY]->(d)`,
    all('SELECT Program_ID id, Program_Name name, Program_Code code, Degree_Level level, Department_ID dept FROM Program')
  );

  counts.Course = await unwind(
    `CREATE (c:Course {courseId: row.id, code: row.code, title: row.title,
                       name: row.code + ' ' + row.title, credits: row.credits, semester: row.sem})
     WITH c, row
     MATCH (d:Department {deptId: row.dept}) CREATE (c)-[:OWNED_BY]->(d)
     WITH c, row
     MATCH (p:Program {programId: row.prog}) CREATE (c)-[:PART_OF_PROGRAM]->(p)`,
    all(`SELECT Course_ID id, Course_Code code, Course_Name title, Credits credits, Semester sem,
                Department_ID dept, Program_ID prog FROM Course`)
  );

  // People (students and faculty are both :Person, plus a role label)
  const students = all(`SELECT u.User_ID id, u.Full_Name name, u.Email email, u.Account_Status status,
                               s.Registration_Number reg, s.Admission_Year batch, s.Current_Semester sem,
                               s.Program_ID prog
                        FROM Student s JOIN "User" u ON u.User_ID = s.User_ID`);
  counts.Student = await unwind(
    `CREATE (p:Person:Student {userId: row.id, name: row.name, email: row.email, status: row.status,
                               regNo: row.reg, batch: row.batch, semester: row.sem})
     WITH p, row MATCH (pr:Program {programId: row.prog}) CREATE (p)-[:ENROLLED_IN]->(pr)`,
    students
  );
```

Figure 23. import.js: batching helper and creation of the uniqueness constraints.



```
}

async function importFromSqlite() {
  const counts = {};

  counts.School = await unwind(
    'CREATE (:School {schoolId: row.id, name: row.name, code: row.code})',
    all('SELECT School_ID id, School_Name name, School_Code code FROM School')
  );

  counts.Department = await unwind(
    `CREATE (d:Department {deptId: row.id, name: row.name, code: row.code})
     WITH d, row MATCH (s:School {schoolId: row.school}) CREATE (d)-[:PART_OF]->(s)`,
    all('SELECT Department_ID id, Department_Name name, Department_Code code, School_ID school FROM Department')
  );

  counts.Program = await unwind(
    `CREATE (p:Program {programId: row.id, name: row.name, code: row.code, level: row.level})
     WITH p, row MATCH (d:Department {deptId: row.dept}) CREATE (p)-[:OFFERED_BY]->(d)`,
    all('SELECT Program_ID id, Program_Name name, Program_Code code, Degree_Level level, Department_ID dept FROM Program')
  );

  counts.Course = await unwind(
    `CREATE (c:Course {courseId: row.id, code: row.code, title: row.title,
                       name: row.code + ' ' + row.title, credits: row.credits, semester: row.sem})
     WITH c, row
     MATCH (d:Department {deptId: row.dept}) CREATE (c)-[:OWNED_BY]->(d)
     WITH c, row
     MATCH (p:Program {programId: row.prog}) CREATE (c)-[:PART_OF_PROGRAM]->(p)`,
    all(`SELECT Course_ID id, Course_Code code, Course_Name title, Credits credits, Semester sem,
                Department_ID dept, Program_ID prog FROM Course`)
  );

  // People (students and faculty are both :Person, plus a role label)
  const students = all(`SELECT u.User_ID id, u.Full_Name name, u.Email email, u.Account_Status status,
                               s.Registration_Number reg, s.Admission_Year batch, s.Current_Semester sem,
                               s.Program_ID prog
                        FROM Student s JOIN "User" u ON u.User_ID = s.User_ID`);
  counts.Student = await unwind(
    `CREATE (p:Person:Student {userId: row.id, name: row.name, email: row.email, status: row.status,
                               regNo: row.reg, batch: row.batch, semester: row.sem})
     WITH p, row MATCH (pr:Program {programId: row.prog}) CREATE (p)-[:ENROLLED_IN]->(pr)`,
    students
  );
```

Figure 24. import.js: importFromSqlite() reads SQLite tables and creates nodes and relationships.

The derived relationship is created by one Cypher statement. It looks for pairs of people who point to the same club, community, project, course or class, counts the shared items, and keeps pairs that share at least two:

```
MATCH (a:Person)-[:MEMBER_OF|WORKS_ON|TAKES|TEACHES|REGISTERED_IN|TEACHES_CLASS]->(x)
      <-[:MEMBER_OF|WORKS_ON|TAKES|TEACHES|REGISTERED_IN|TEACHES_CLASS]-(b:Person)
WHERE a.userId < b.userId
WITH a, b, count(DISTINCT x) AS shared, collect(DISTINCT x.name)[..4] AS via
WHERE shared >= 2
MERGE (a)-[c:CONNECTED]->(b) SET c.shared = shared, c.via = via
```

6.3 Running the import

The terminal output below is from npm run graph:import. It lists how many nodes and relationships were created from each part of the SQLite data, and how many CONNECTED relationships were derived.

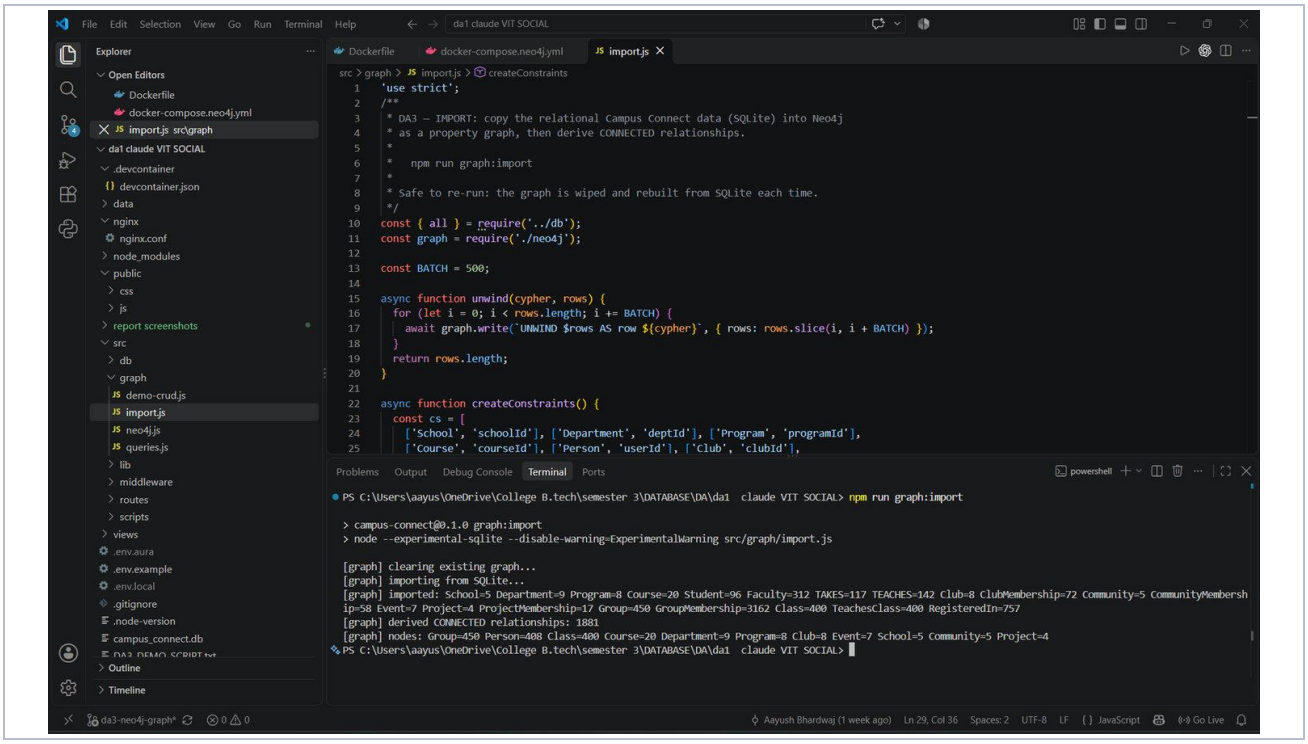


Figure 25. Terminal: npm run graph:import. The graph is cleared, SQLite is imported, and 1,881 CONNECTED relationships are derived.

6.4 The imported graph

After the import, Neo4j Desktop reports the size of the database in its Database information panel, and the contents can be explored in the Query tab.

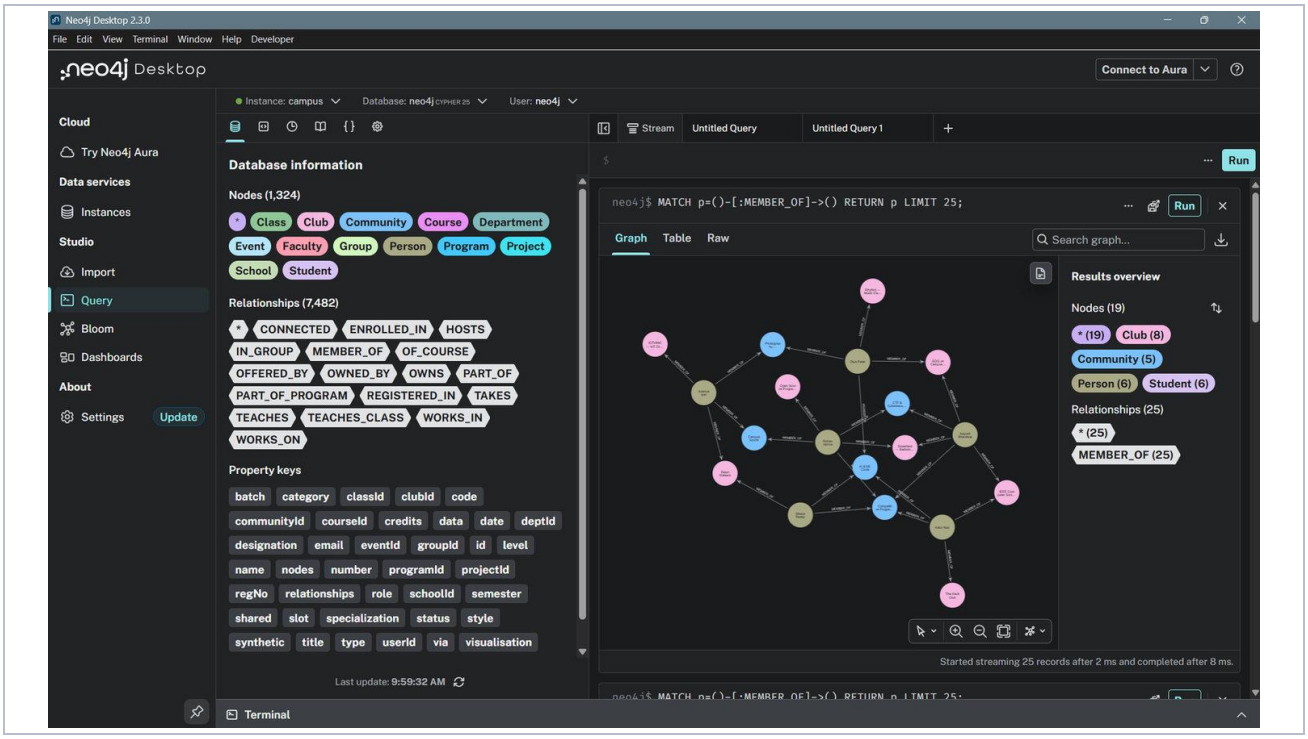


Figure 26. Neo4j Desktop: the database holds 1,324 nodes and 7,482 relationships; the labels and relationship types are listed on the left.

A node can be opened as a table to see its properties and its relationships, which shows how a row of the Student table became a node with properties and a MEMBER_OF relationship to a Club.

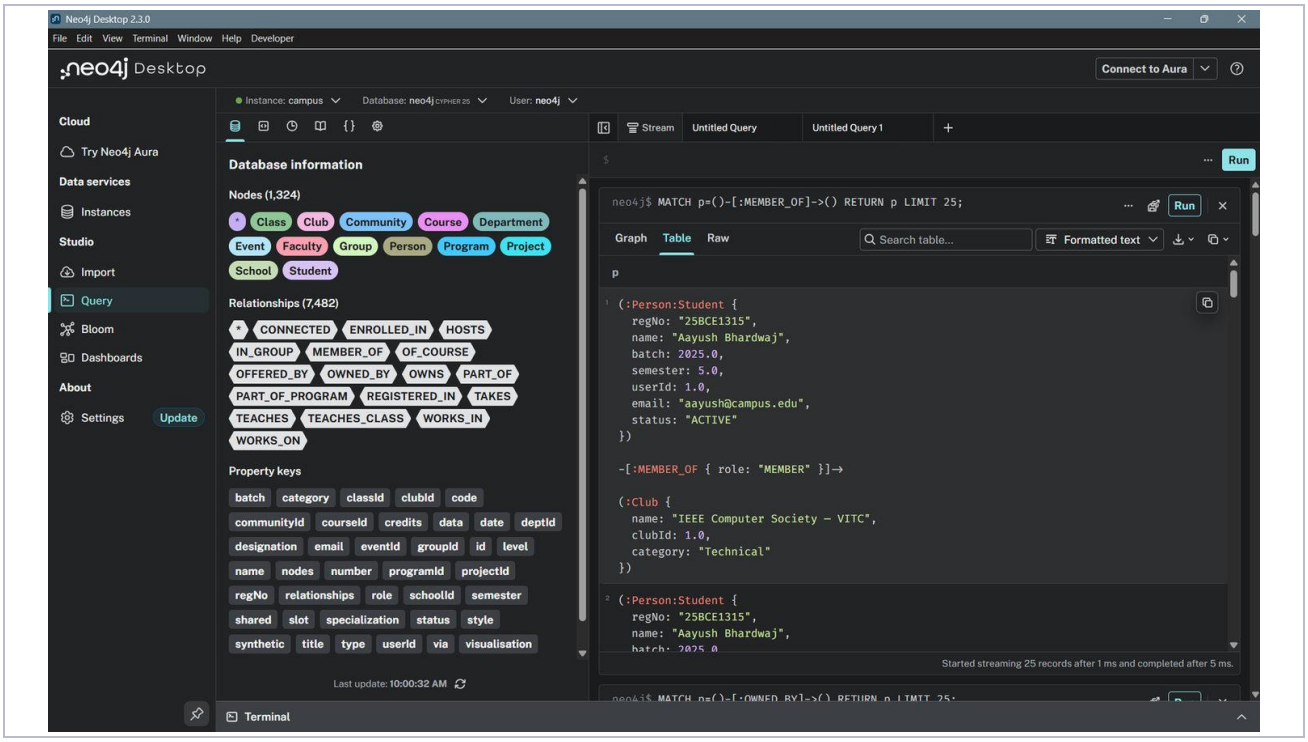


Figure 27. A Student node with its properties and one of its MEMBER_OF relationships (table view).

The CONNECTED relationships, created by the import, can be displayed as a graph.

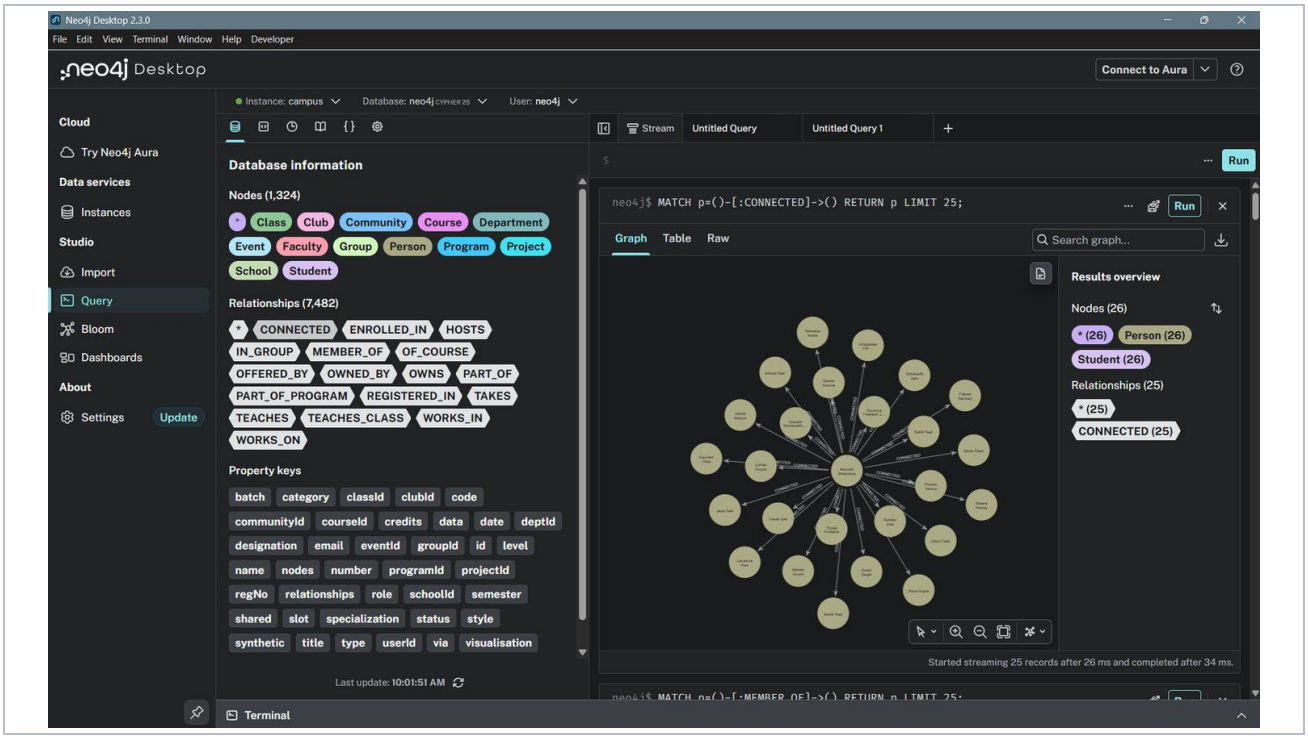


Figure 28. CONNECTED relationships between people (graph view).

Contents of the graph database

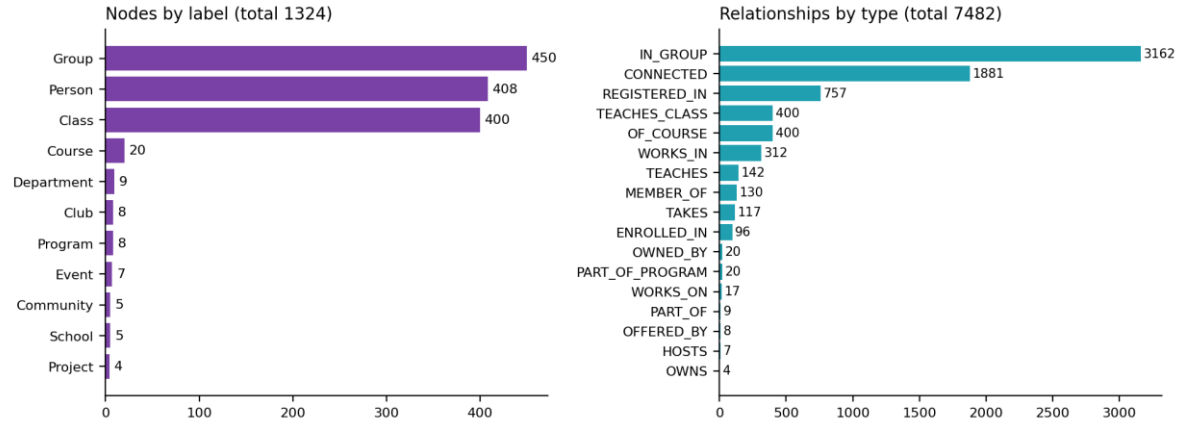


Figure 29. Contents of the graph database: nodes by label and relationships by type.

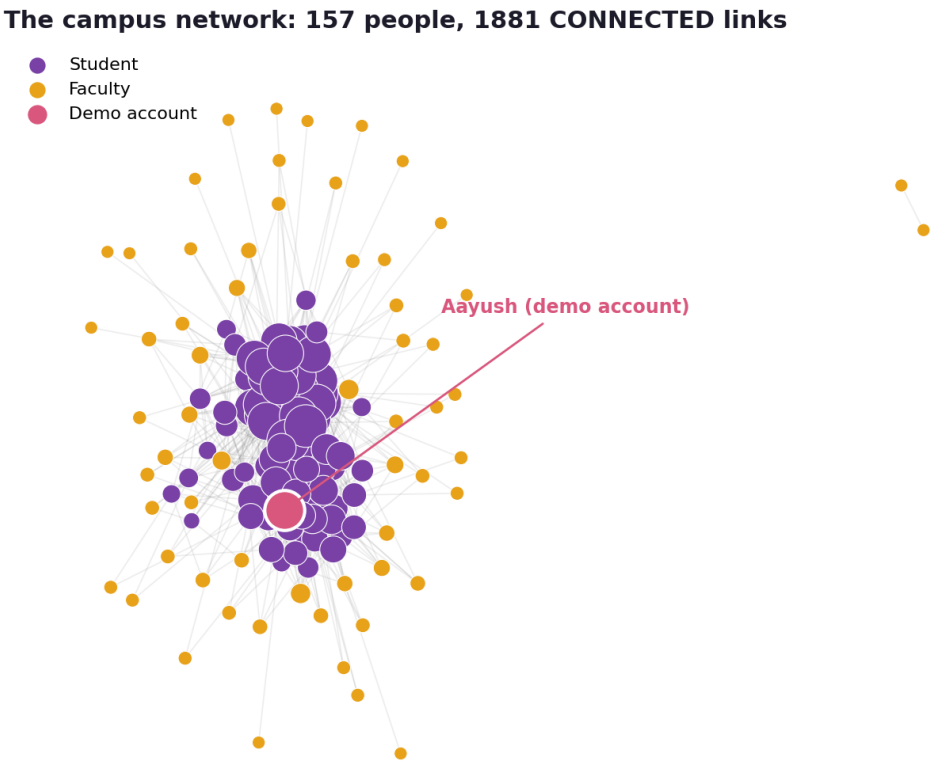


Figure 30. The campus network: every person with at least one CONNECTED relationship. Students form a dense core; faculty connect through the classes they teach.

157 people have at least one CONNECTED relationship, joined by 1,881 relationships. All 96 students are connected. A teacher is connected only if they share at least two classes or other items with someone, so some faculty are present in the graph (with their department and classes) but not in the people-to-people network.

7. Core Operations (Create, Read, Update, Delete)

The four basic operations were run in Neo4j Browser against the live graph. A test student node with userId 9001 is created, linked to a club, read back, updated, and finally deleted. Each statement is shown with its result. The same sequence is available as a terminal demonstration with npm run graph:demo.

7.1 Create

CREATE adds a node with two labels and two properties. Figure (a) below shows this statement and its result in table view; the status line reads one node created, two properties set and two labels added.

```
CREATE (s:Person:Student {userId: 9001, name: 'Demo Student'})
RETURN s.userId AS userId, s.name AS name
```

The same node can be created together with a relationship in one statement. Figure (b) creates the student and links them to the first club; the status line then also reads one relationship created. This second form is the one used in the steps that follow (the test node was removed between the two runs).

```
CREATE (s:Person:Student {userId: 9001, name: 'Demo Student'})
WITH s MATCH (c:Club) WITH s, c ORDER BY c.clubId LIMIT 1
CREATE (s)-[m:MEMBER_OF {role: 'MEMBER'}]->(c)
RETURN s, m, c
```

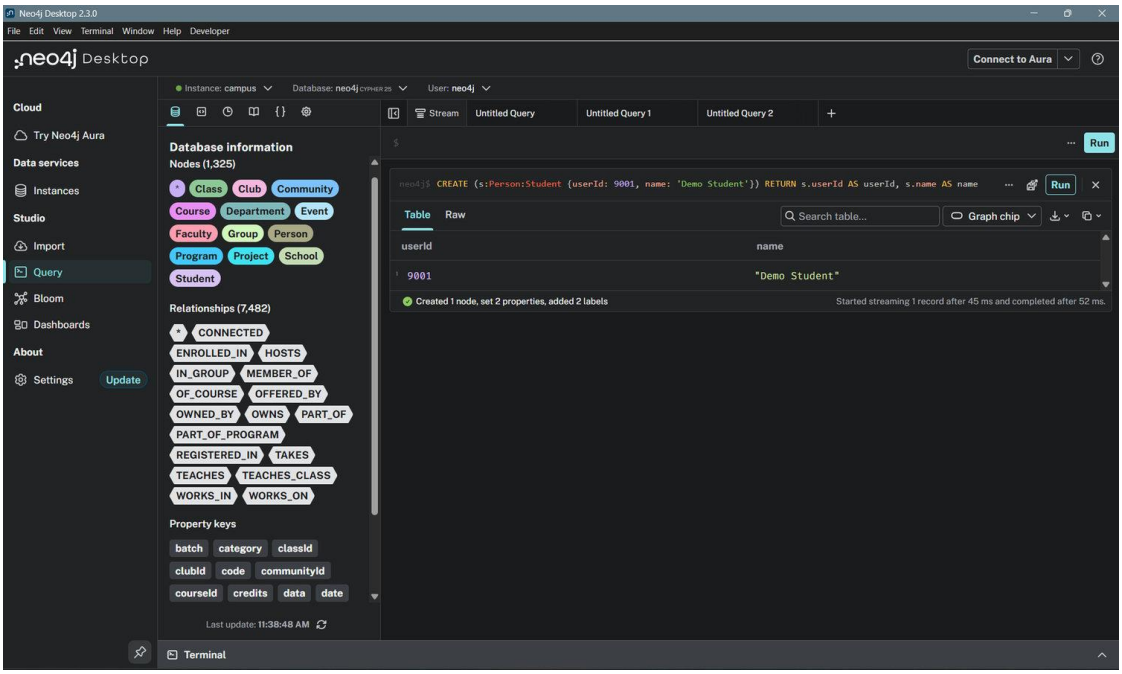


Figure 31(a). CREATE: the node is created (table view).

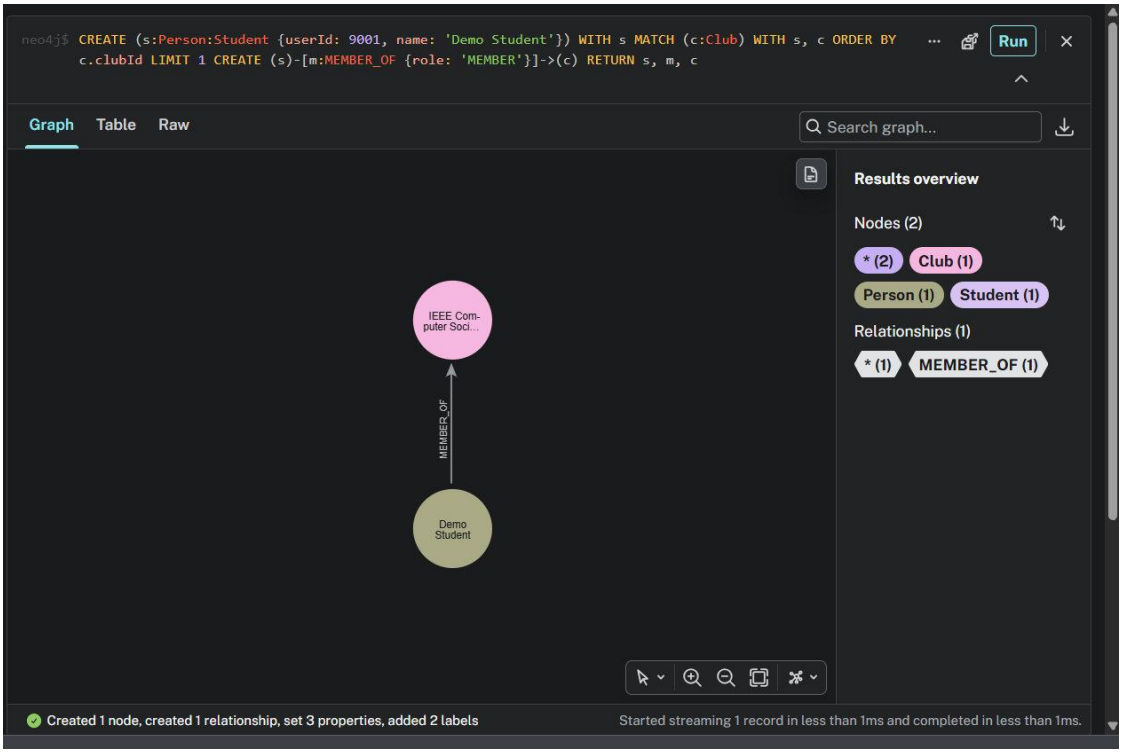


Figure 31(b). CREATE: the node is created and linked to a club (graph view).

7.2 Read

MATCH finds the node by its unique id, using the index that the constraint created, and follows its relationship.

```
MATCH (s:Person {userId: 9001})-[m:MEMBER_OF]->(c:Club) RETURN s, m, c
```

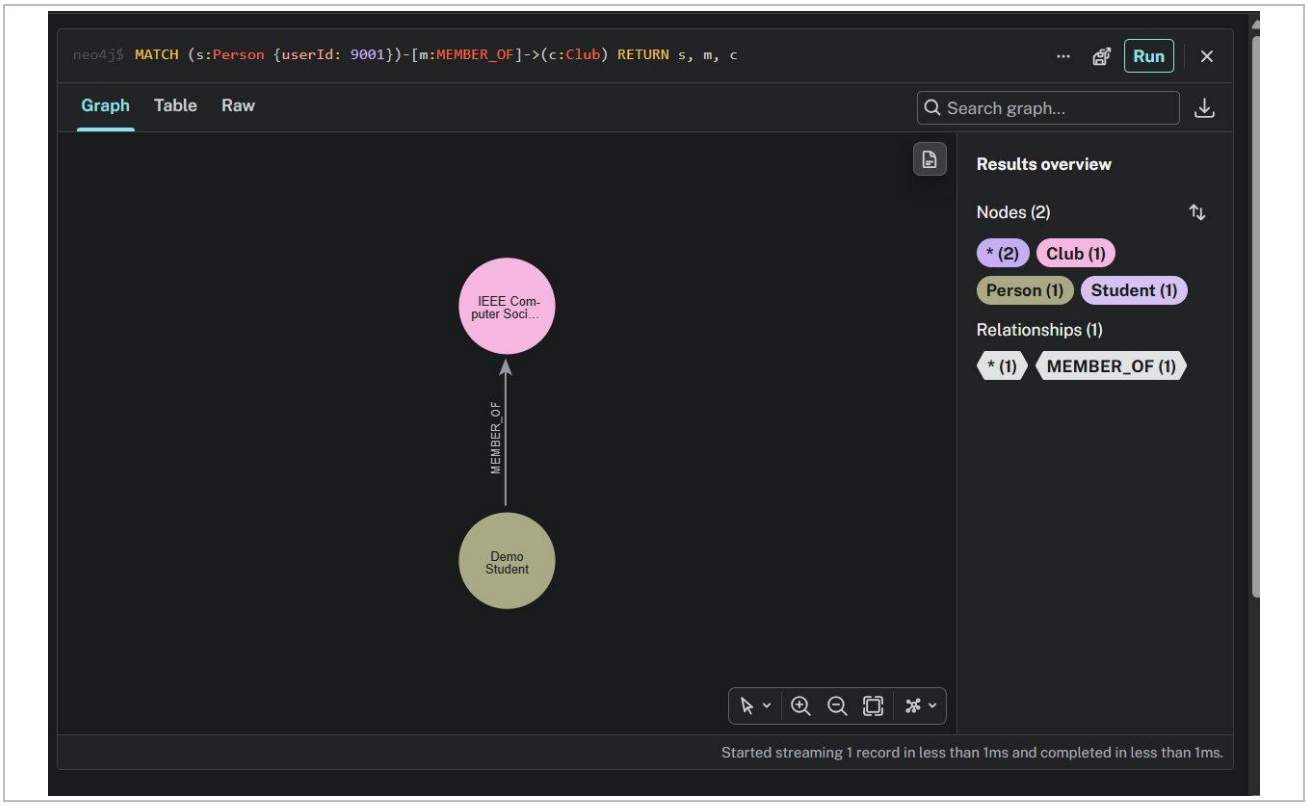


Figure 32. READ: the node is found by its id together with its club.

7.3 Update

SET changes a property of the node and a property of the relationship. The node and its relationships stay in place.

```
MATCH (s:Person {userId: 9001})-[m:MEMBER_OF]->(c:Club)
SET s.name = 'Demo Student (updated)', m.role = 'CORE'
RETURN s, m, c
```

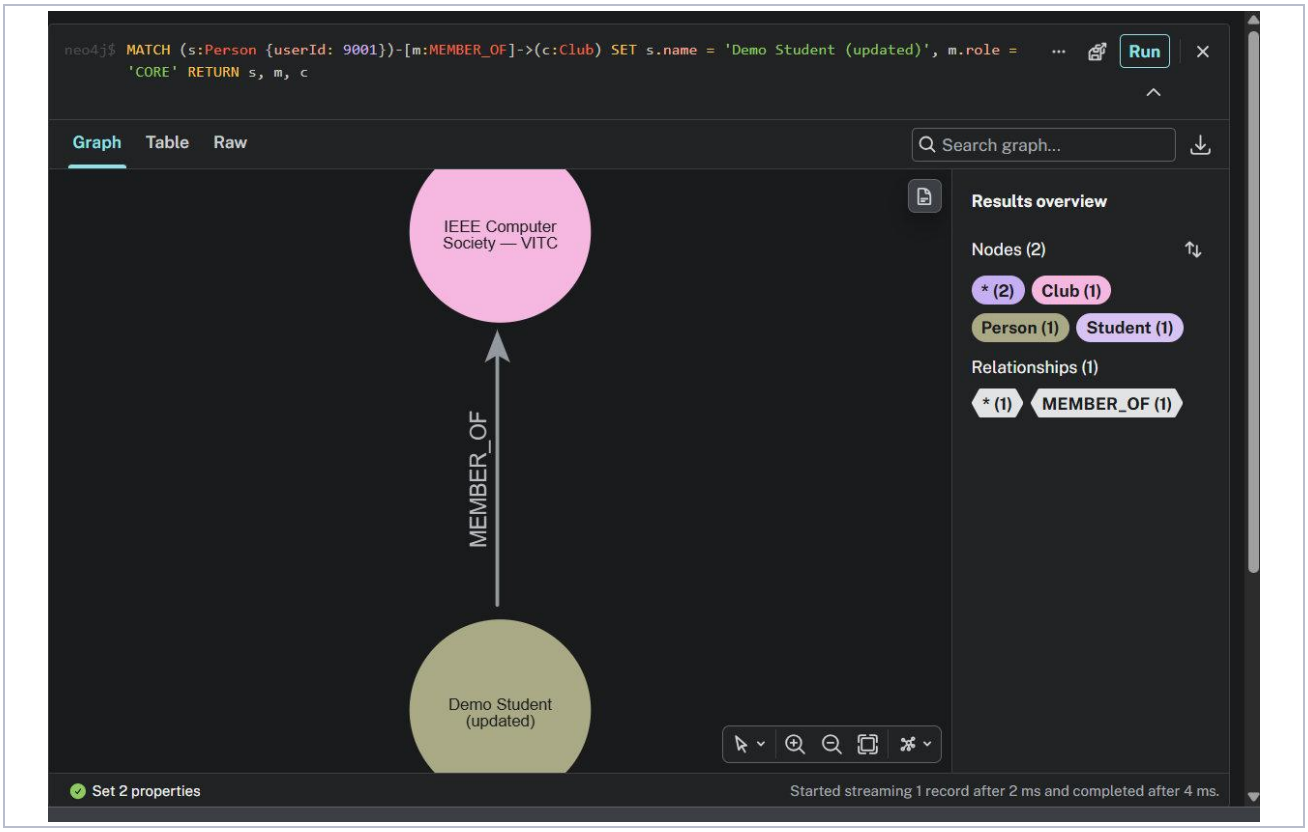


Figure 33. UPDATE: the student name is changed; the status line reads "Set 2 properties".

7.4 Delete

DETACH DELETE removes the node together with every relationship attached to it. In Neo4j a node that still has relationships cannot be deleted without DETACH.

```
MATCH (s:Person {userId: 9001}) DETACH DELETE s
```



Figure 34. DELETE: one node and one relationship are deleted.

A final read confirms that the node is gone.

```
MATCH (s:Person {userId: 9001}) RETURN count(s) AS remaining
```



Figure 35. Verification: remaining = 0.

8. Technology-Specific Queries

This section demonstrates queries that use the strengths of a graph database: traversal over several relationship types, multi-hop patterns, shortest paths, filtering on paths and on aggregates, and recommendation. All queries were run in Neo4j Browser for the demonstration account, Aayush Bhardwaj (userId 1). Each query is shown with its Cypher, a short explanation, and its result as a table and as a graph.

#	Query	Graph feature shown
8.1	People who share the most with you	Several relationship types in one pattern
8.2	People you may know	Two-hop traversal with exclusion
8.3	Clubs worth a look	Recommendation through connections
8.4	Shortest path between two people	shortestPath
8.5	Best-connected people	Degree centrality
8.6	Clubs with the most shared members	Pattern between two nodes of one label
8.7	Who can introduce me?	Filtering on path length and on path members
8.8	Study buddies	Filter on an aggregate, then a second pattern
8.9	Clubmates I am not connected to	Negative pattern (NOT)
8.10	How can I reach this person?	Path, with the reason for each step

8.1 People who share the most with you

The pattern starts at the student, follows any of six relationship types to a shared node (a club, course, class and so on) and comes back to another person. Counting the distinct shared nodes ranks people by how much they have in common. In SQL this is a union of several joins.

```
MATCH (me:Person {userId: 1})-[:MEMBER_OF|WORKS_ON|TAKES|TEACHES|REGISTERED_IN|TEACHES_CLASS]->(x)
      <-[:MEMBER_OF|WORKS_ON|TAKES|TEACHES|REGISTERED_IN|TEACHES_CLASS]-(o:Person)
WHERE o <> me
RETURN o.name AS name, count(DISTINCT x) AS shared,
       collect(DISTINCT x.name)[..4] AS via
ORDER BY shared DESC, name LIMIT 8
```

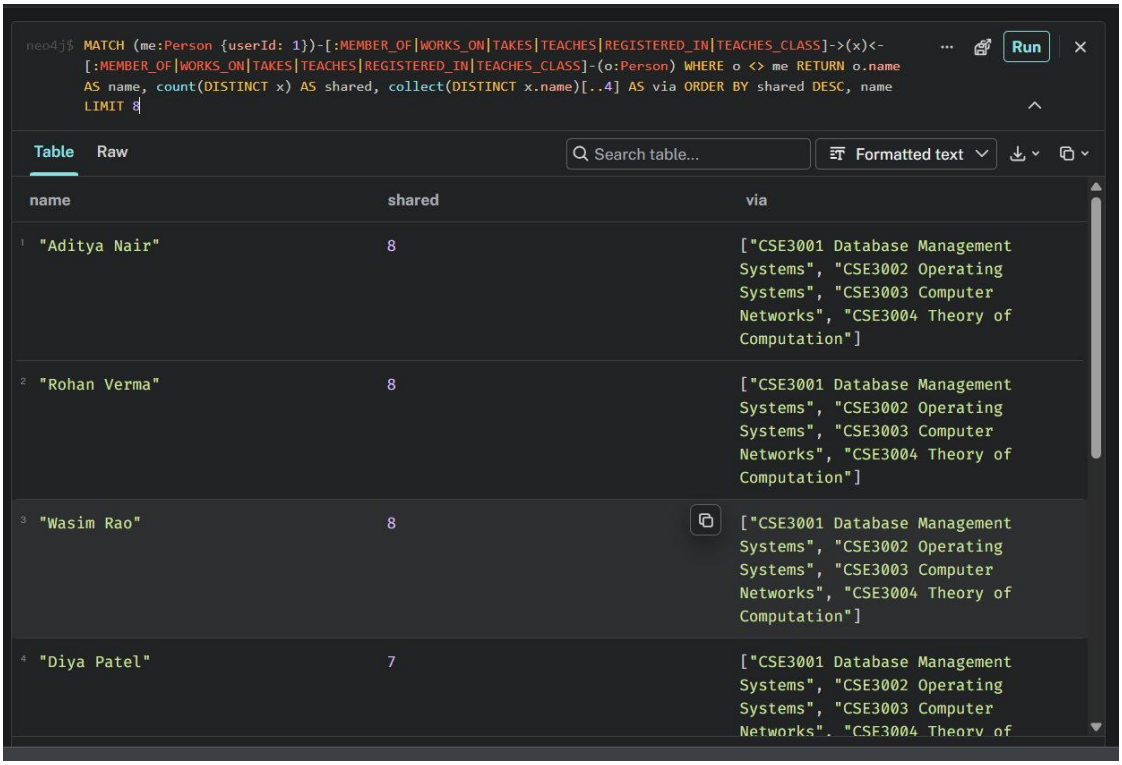


Figure 36(a). Query 8.1: ranked people with the shared items (table view).

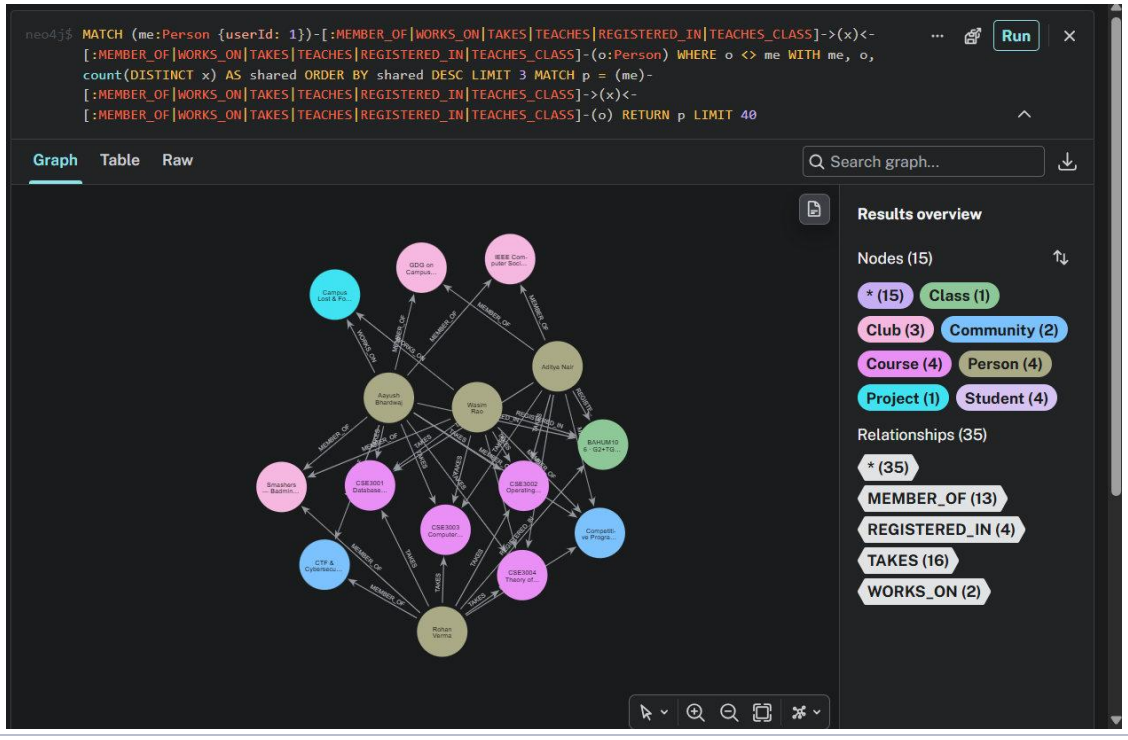


Figure 36(b). Query 8.1: the three closest people and the shared courses, clubs and classes between them and the student (graph view).

Result: Aditya Nair (8 shared), Rohan Verma (8 shared), Wasim Rao (8 shared) are the closest people. They share the same core semester courses.

8.2 People you may know

A two-hop traversal: friends of friends who are not yet connected to the student, ranked by the number of mutual connections. The NOT pattern in the WHERE clause removes people the student is already connected to.

```
MATCH (me:Person {userId: 1})-[:CONNECTED]-(f:Person)-[:CONNECTED]-(fof:Person)
WHERE fof <> me AND NOT (me)-[:CONNECTED]-(fof)
RETURN fof.name AS name, count(DISTINCT f) AS mutual,
       collect(DISTINCT f.name)[..3] AS through
ORDER BY mutual DESC, name LIMIT 8
```

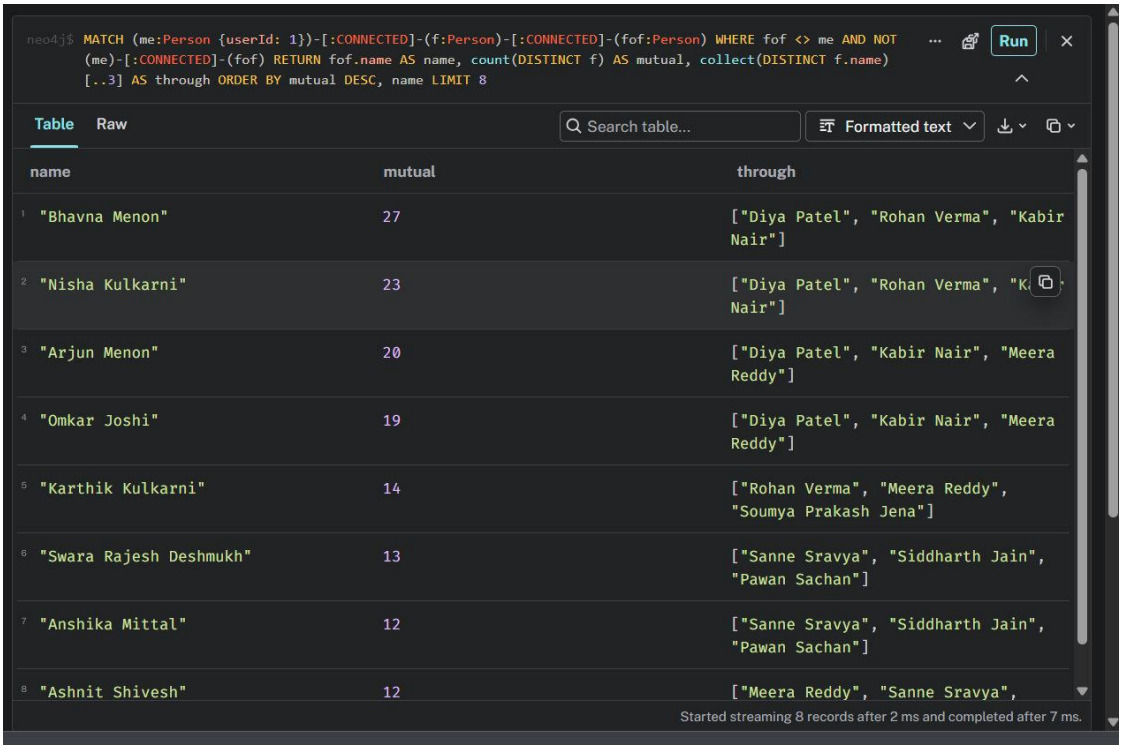


Figure 37(a). Query 8.2: suggested people with their mutual connections (table view).

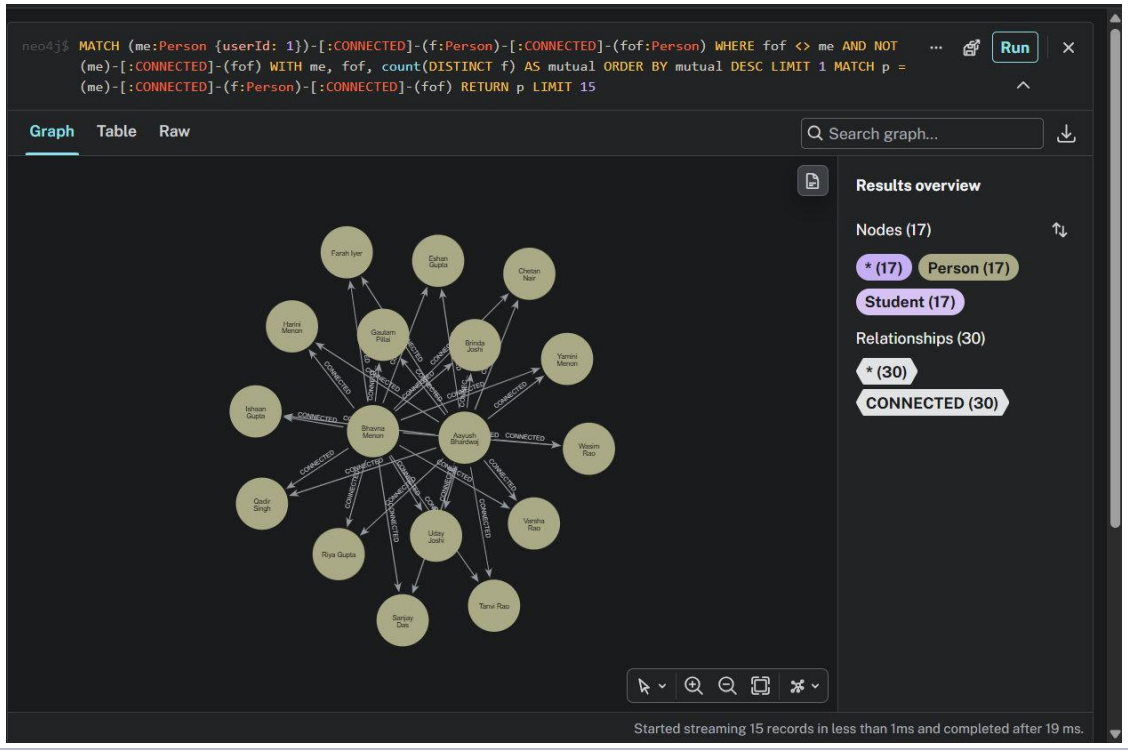


Figure 37(b). Query 8.2: the top suggestion and the mutual connections that link them to the student (graph view).

Result: the top suggestion has 27 mutual connections, followed by people with 23, 20 and 19.

8.3 Clubs worth a look

A recommendation: clubs that the student's connections belong to and the student has not joined. The count shows how many connections are members.

```
MATCH (me:Person {userId: 1})-[:CONNECTED]-(f:Person)-[:MEMBER_OF]->(c:Club)
WHERE NOT (me)-[:MEMBER_OF]->(c)
RETURN c.name AS club, c.category AS category, count(DISTINCT f) AS connections
ORDER BY connections DESC, club LIMIT 6
```

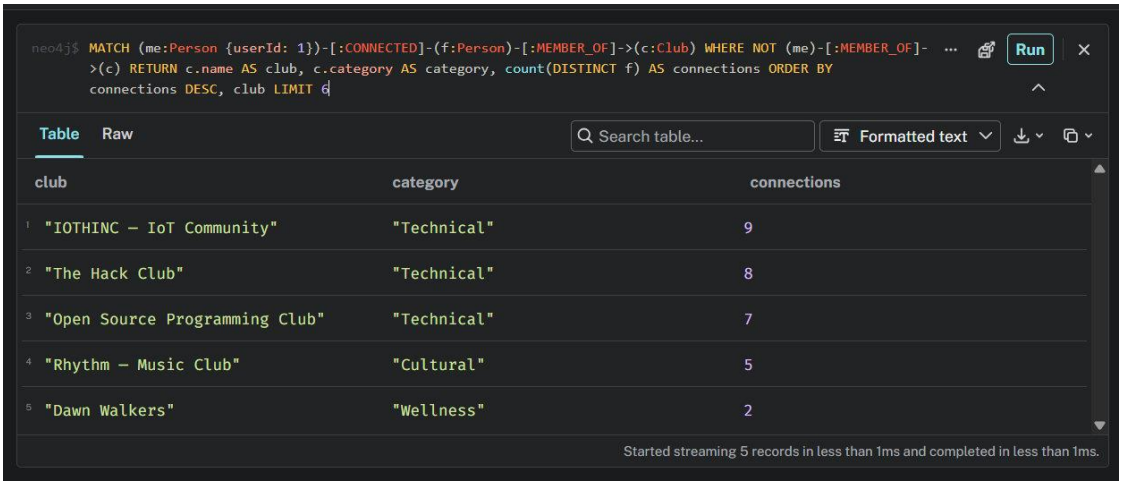


Figure 38(a). Query 8.3: recommended clubs (table view).

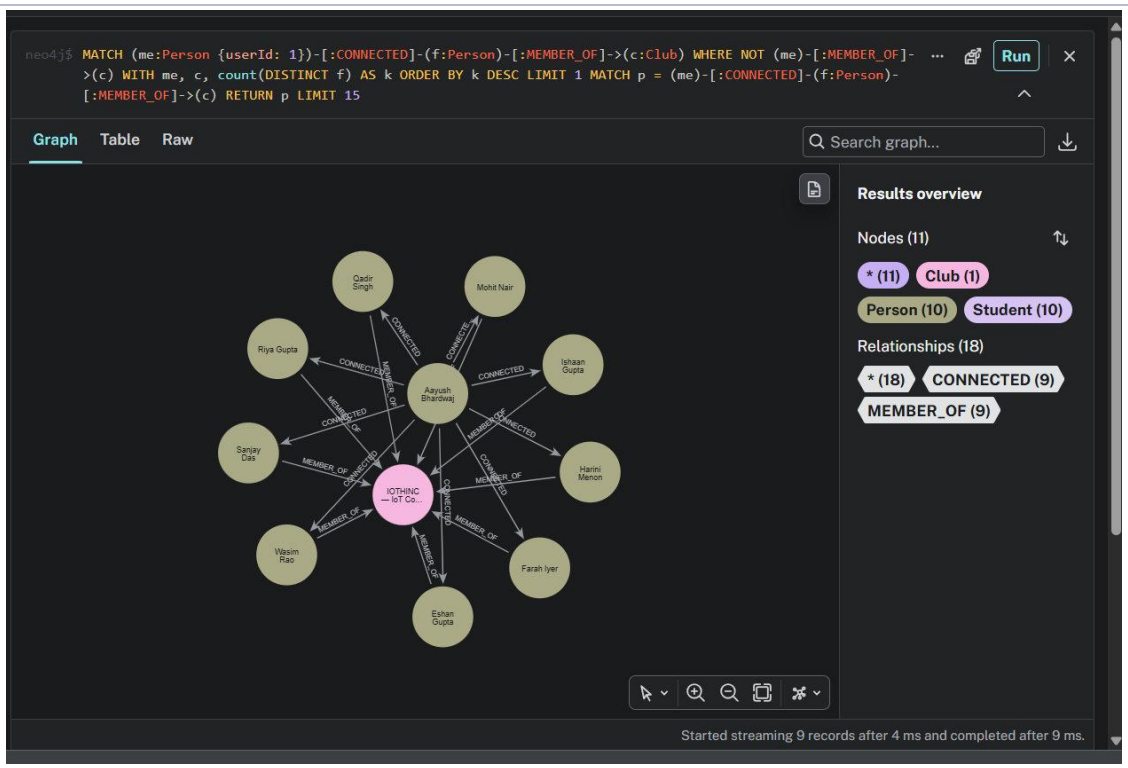


Figure 38(b). Query 8.3: the top club and the connections who are members (graph view).

Result: IOTHINC, the IoT community, is recommended first (9 of the student's connections are members), then The Hack Club (8) and the Open Source Programming Club (7).

8.4 Shortest path between two people

shortestPath finds the shortest chain of CONNECTED relationships between two people, up to six hops. This is the "how many introductions separate us" question, which needs a recursive query in SQL.

```
MATCH (a:Person {userId: 1}), (b:Person {name: 'Bhavna Menon'}),
      p = shortestPath((a)-[:CONNECTED*..6]-(b))
RETURN [n IN nodes(p) | n.name] AS people, length(p) AS hops
```

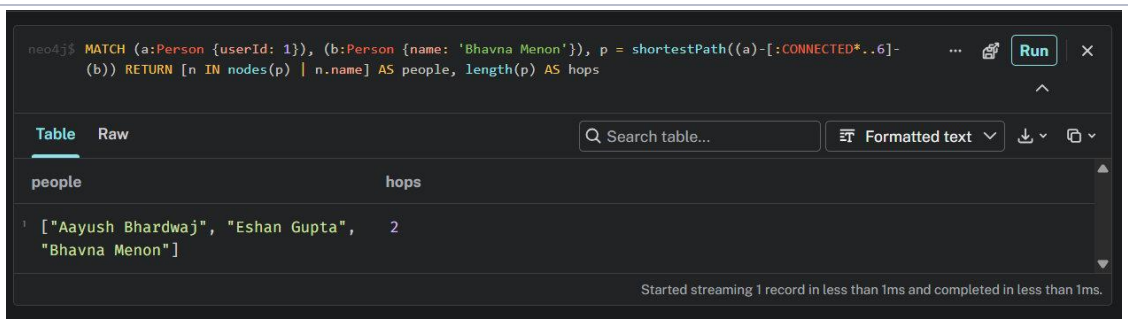


Figure 39(a). Query 8.4: the path and its length (table view).

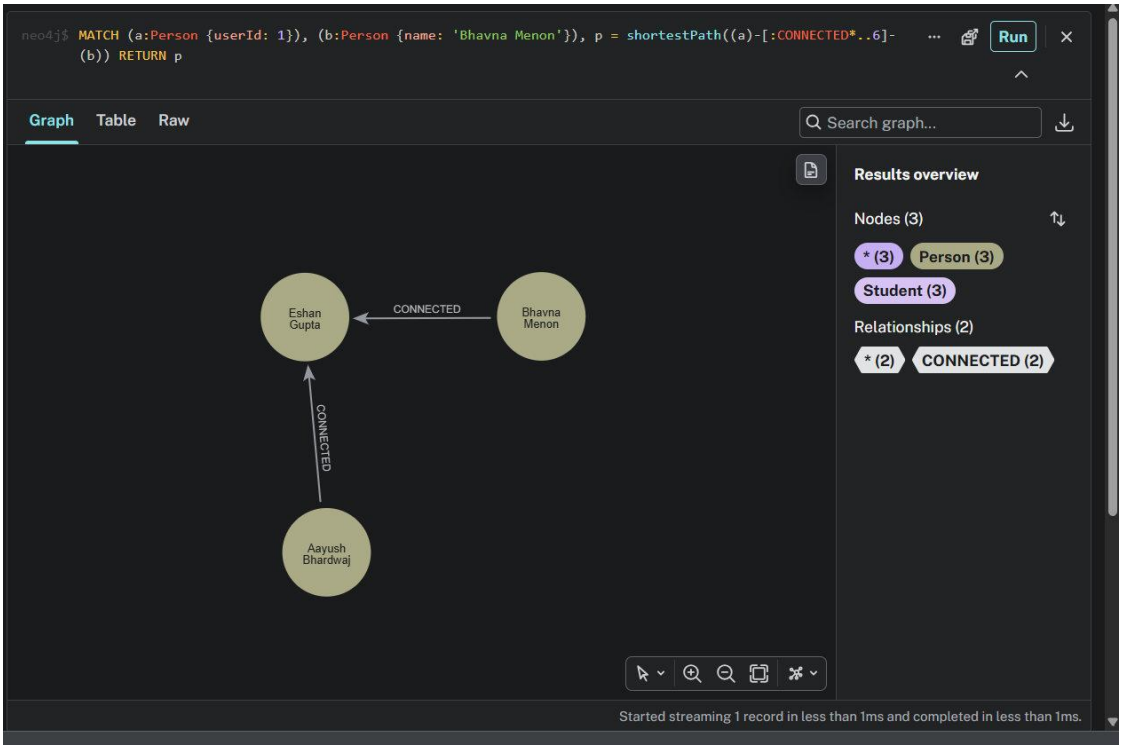


Figure 39(b). Query 8.4: the path as a graph: Aayush Bhardwaj, Eshan Gupta, Bhavna Menon.

Result: two hops. Aayush Bhardwaj is connected to Eshan Gupta, who is connected to Bhavna Menon.

8.5 Best-connected people

Degree centrality: the number of CONNECTED relationships of each person. People with many connections are the hubs of the campus network.

```
MATCH (p:Person)-[c:CONNECTED]-()
RETURN p.name AS name, count(c) AS connections
ORDER BY connections DESC, name LIMIT 6
```

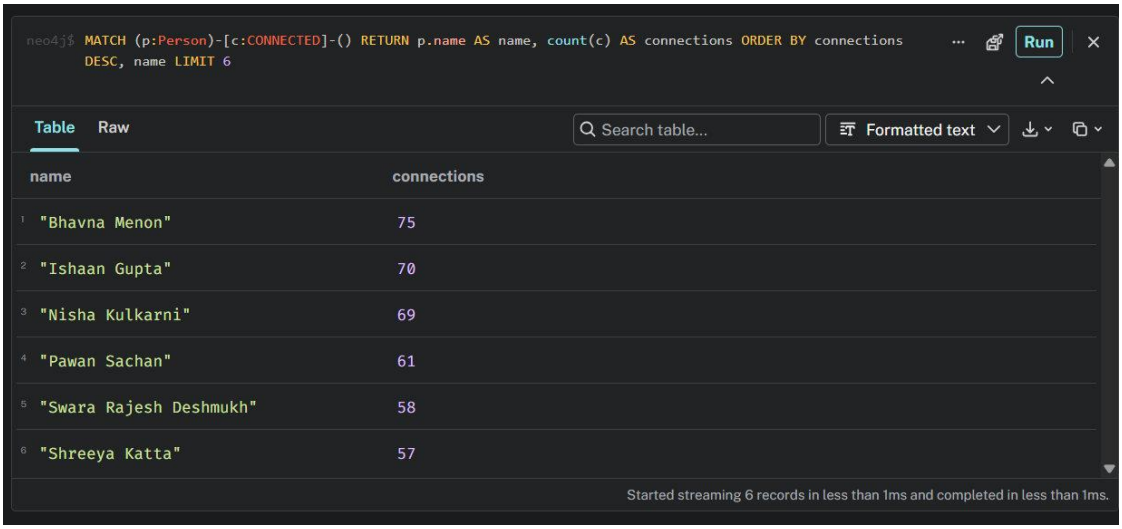


Figure 40(a). Query 8.5: the most connected people (table view).

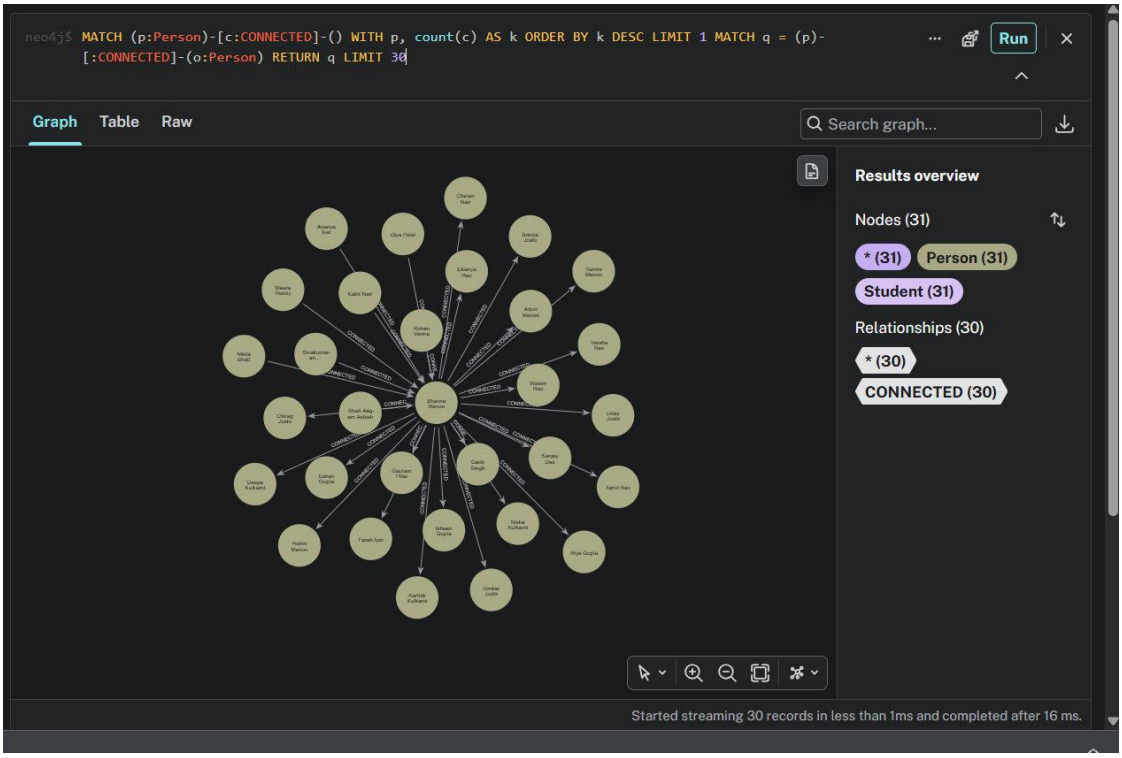


Figure 40(b). Query 8.5: the best-connected person and their connections (graph view).

Result: the best-connected people have 75, 70 and 69 connections.

8.6 Clubs with the most shared members

Pairs of clubs whose members overlap. This is useful when two clubs want to organise a joint event. The condition a.clubId < b.clubId lists each pair once.

```
MATCH (a:Club)<-[:MEMBER_OF]-(p:Person)-[:MEMBER_OF]->(b:Club)
WHERE a.clubId < b.clubId
RETURN a.name AS clubA, b.name AS clubB, count(DISTINCT p) AS shared
ORDER BY shared DESC LIMIT 5
```

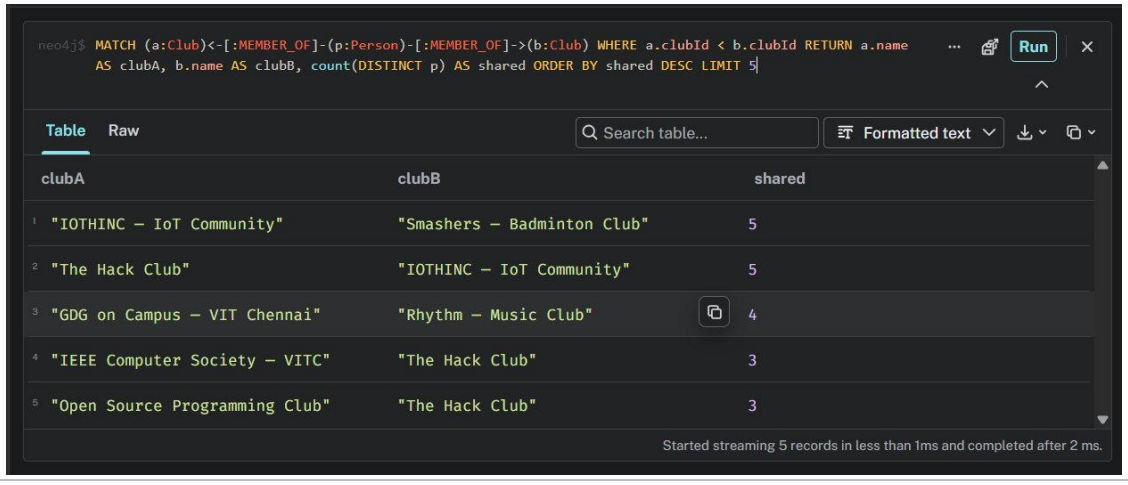


Figure 41(a). Query 8.6: club pairs ranked by shared members (table view).

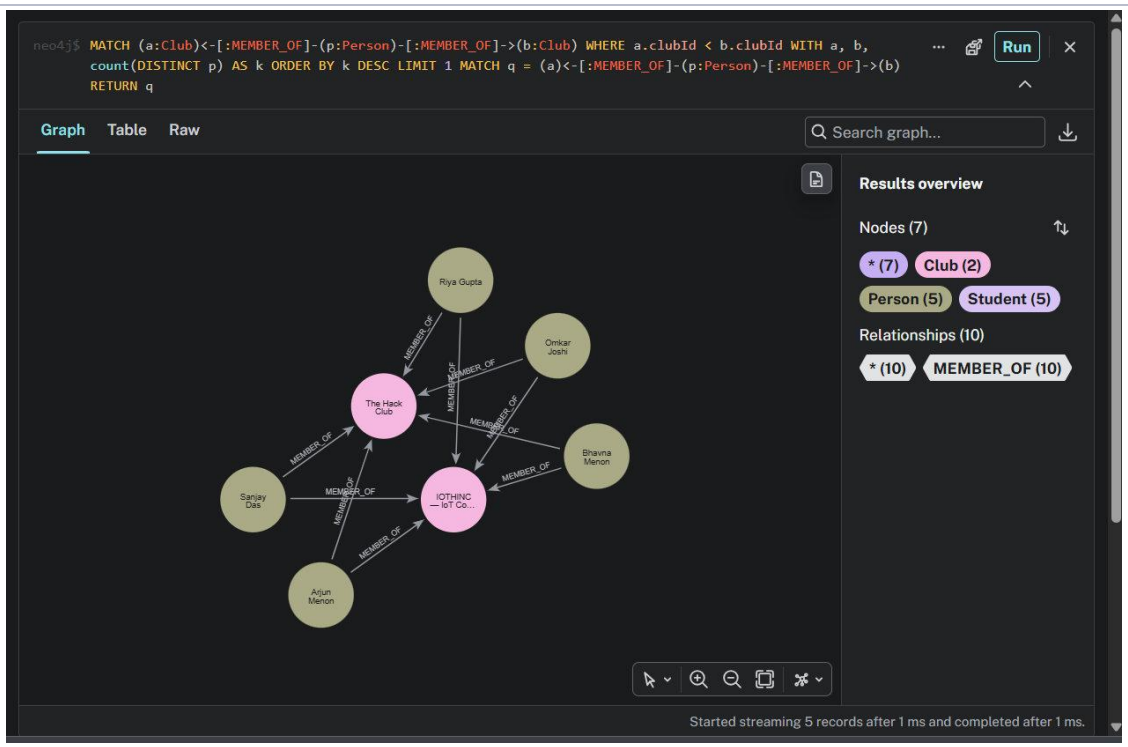


Figure 41(b). Query 8.6: the two clubs with the most shared members and those members (graph view).

Result: IOTHINC with Smashers Badminton Club, and The Hack Club with IOTHINC, each share 5 members.

8.7 Who can introduce me? (filtering on path length)

This query uses only students. For every student the demonstration account is not yet connected to, it finds the shortest chain of connections, keeps only chains in which every person is a student (the all() predicate), and lists the people in the middle who would introduce the student. Sorting by hops shows the farthest students first.

```
MATCH (a:Student {userId: 1}), (b:Student)
WHERE b <> a AND NOT (a)-[:CONNECTED]-(b)
MATCH p = shortestPath((a)-[:CONNECTED*..4]-(b))
WHERE all(n IN nodes(p) WHERE n:Student)
WITH b, p, length(p) AS hops
RETURN b.name AS person, hops,
       [n IN nodes(p)[1..-1] | n.name] AS introducedBy
ORDER BY hops DESC, person LIMIT 10
```

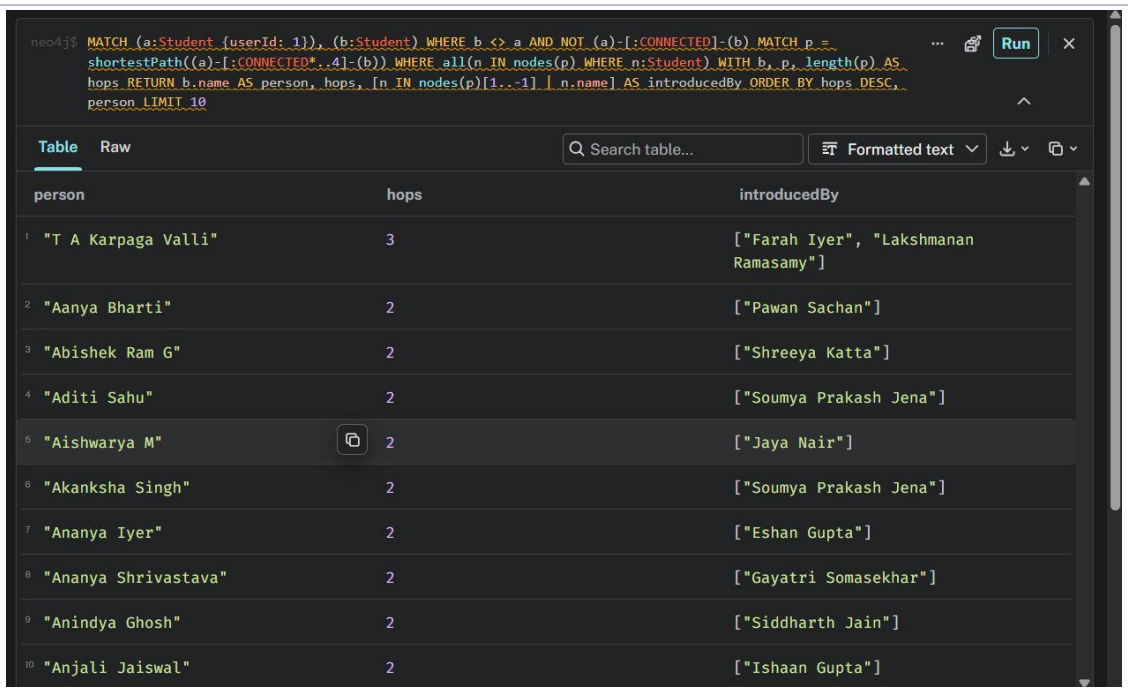


Figure 42(a). Query 8.7: students, their distance and who would introduce them (table view).

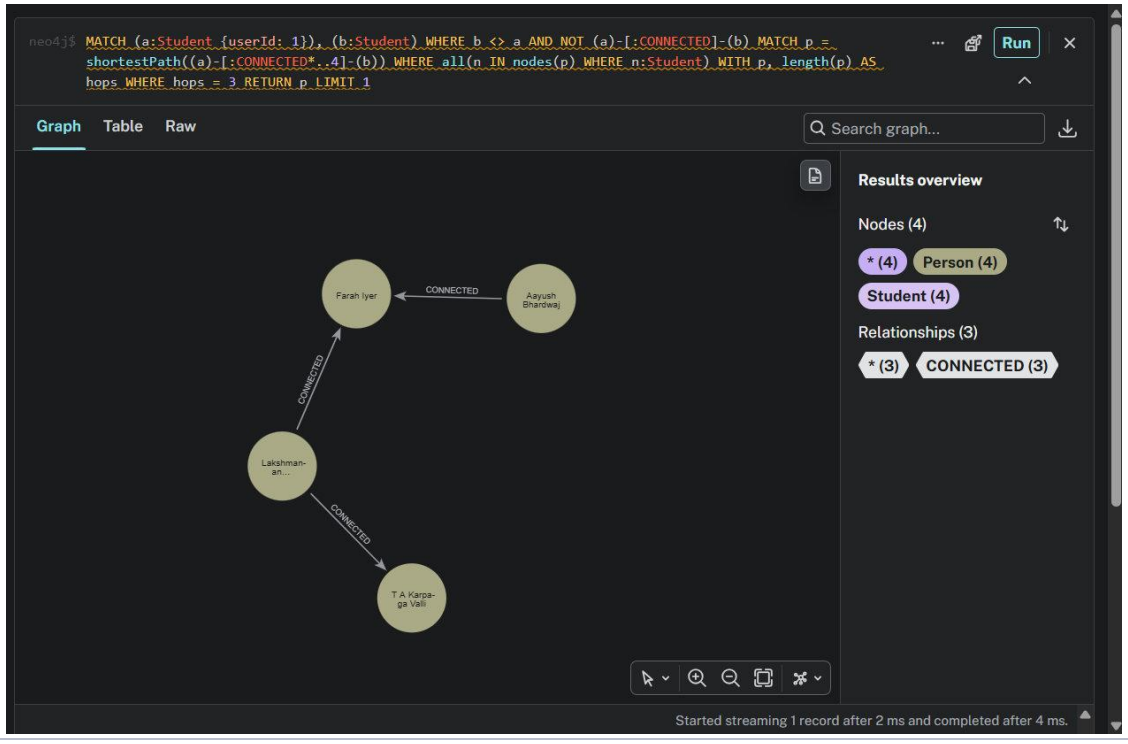


Figure 42(b). Query 8.7: a three-hop chain between the student and a distant student (graph view).

Result: one student is three hops away, introduced through two other students; the others are two hops away, each with a single introducer.

8.8 Study buddies (filter on an aggregate)

Students who share at least two registered classes with the demonstration account (the filter on the count) and also belong to a common club. A second pattern is matched only for the students that pass the first filter.

```
MATCH (me:Student {userId: 1})-[:REGISTERED_IN]->(c:Class)<-[:REGISTERED_IN]-(o:Student)
WITH me, o, count(c) AS sharedClasses WHERE sharedClasses >= 2
MATCH (me)-[:MEMBER_OF]->(club:Club)<-[:MEMBER_OF]-(o)
RETURN o.name AS student, sharedClasses, collect(club.name) AS sharedClubs
ORDER BY sharedClasses DESC, student LIMIT 10
```

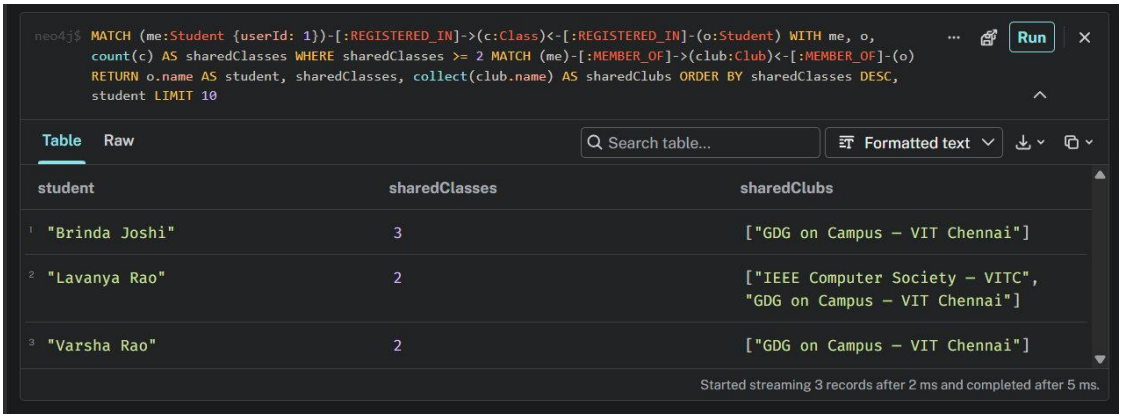


Figure 43(a). Query 8.8: study buddies with shared classes and clubs (table view).

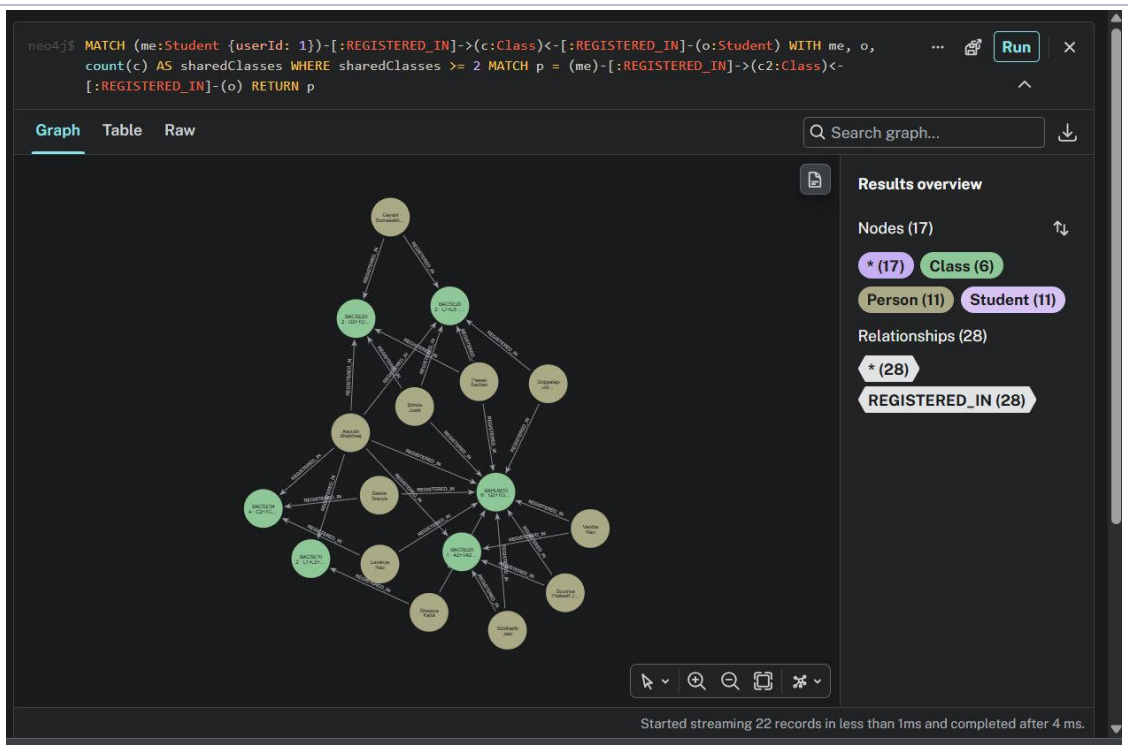


Figure 43(b). Query 8.8: the classes the student shares with them (graph view).

Result: three students share two or more classes and a club with the demonstration account; one of them shares three classes and the GDG on Campus club.

8.9 Clubmates I am not connected to (negative pattern)

Students who are in a club or community with the demonstration account but are not yet connected to it. The NOT (me)-[:CONNECTED]-(o) pattern excludes existing connections.

```
MATCH (me:Student {userId: 1})-[:MEMBER_OF]->(c)<-[:MEMBER_OF]-(o:Student)
WHERE NOT (me)-[:CONNECTED]-(o)
RETURN o.name AS student, collect(DISTINCT c.name) AS inCommon
ORDER BY student LIMIT 10
```

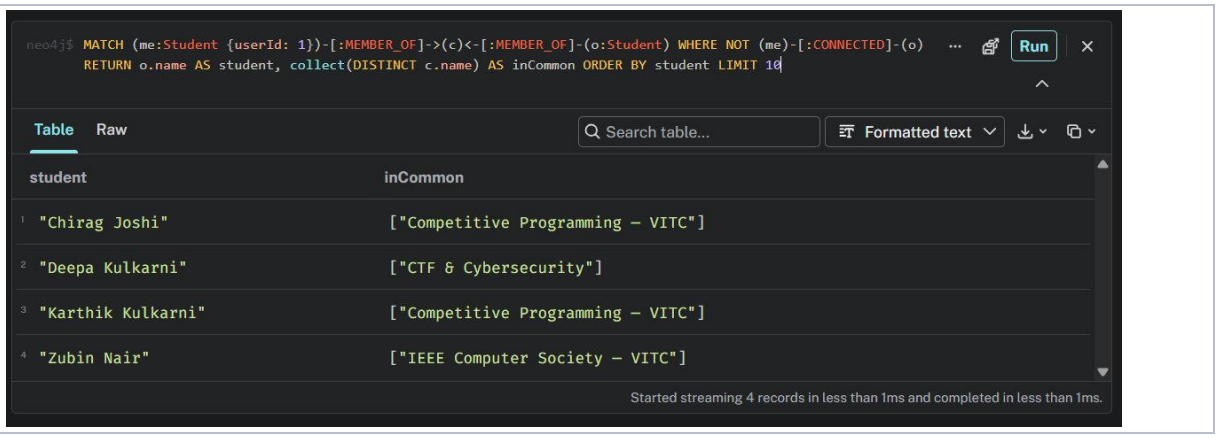


Figure 44(a). Query 8.9: students in a shared club or community (table view).

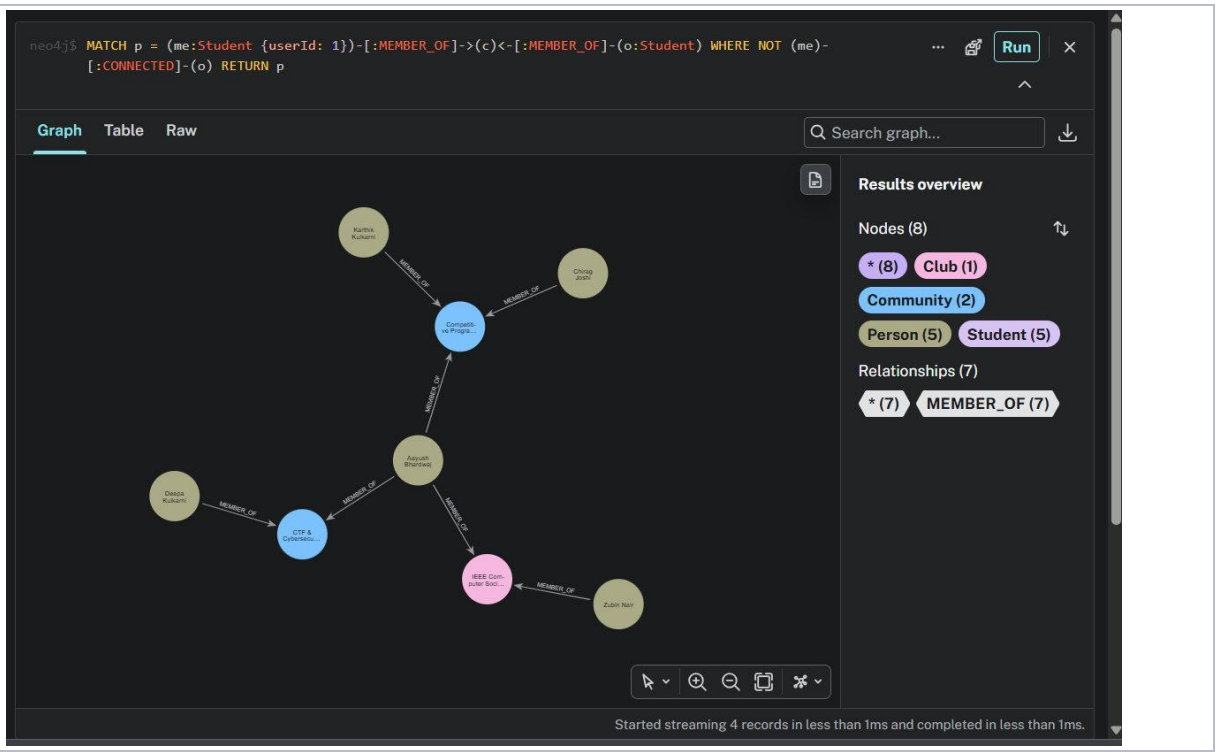


Figure 44(b). Query 8.9: the shared clubs and communities that link them (graph view).

Result: four students share a club with the demonstration account without being connected to it, in Competitive Programming, CTF and Cybersecurity, and IEEE Computer Society.

8.10 How can I reach this person?

The last query explains a route step by step. It finds the shortest student-only path to a chosen person, then unwinds it into one row per step, together with the reason for each link (the shared items stored on the CONNECTED relationship).

```
MATCH (a:Student {userId: 1}), (b:Student {name: 'T A Karpaga Valli'}),
      p = shortestPath((a)-[:CONNECTED*..6]-(b))
WHERE all(n IN nodes(p) WHERE n:Student)
UNWIND range(0, length(p) - 1) AS i
RETURN i + 1 AS step, nodes(p)[i].name AS fromPerson,
       nodes(p)[i + 1].name AS toPerson, relationships(p)[i].via AS because
```

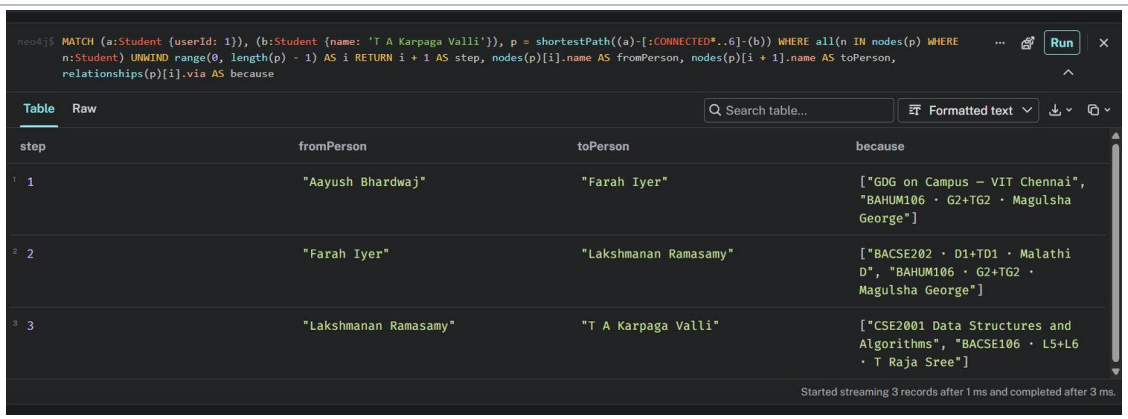


Figure 45(a). Query 8.10: each step of the route with the reason for the link (table view).

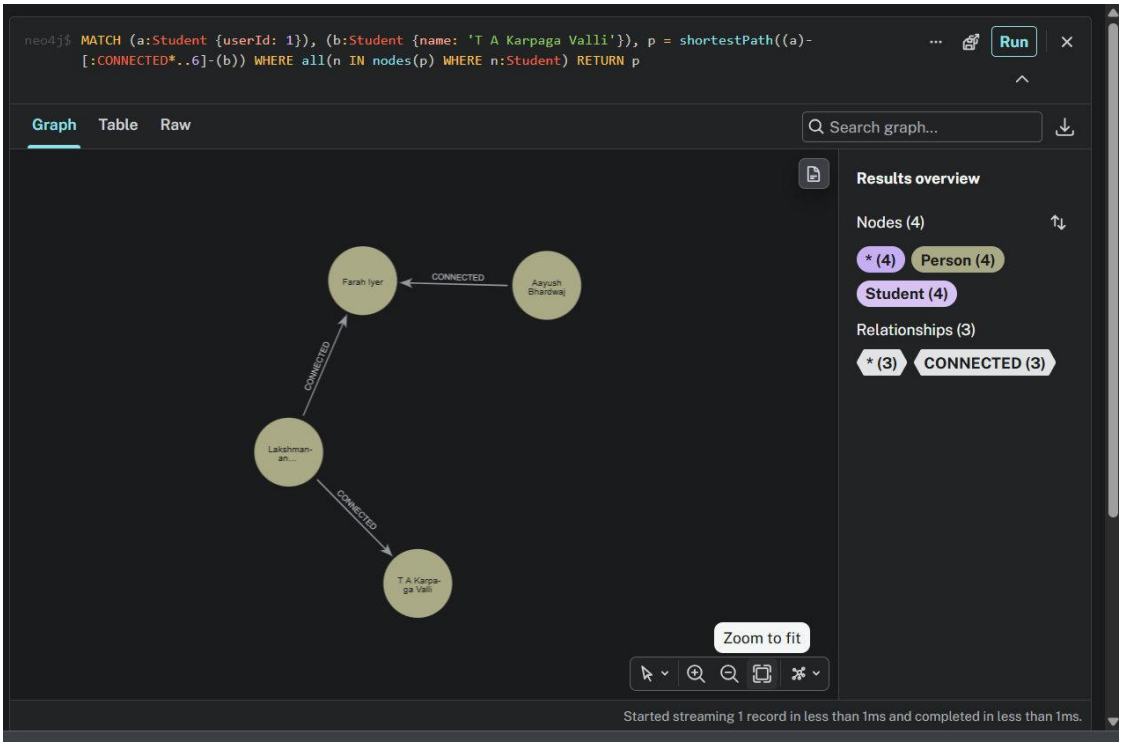


Figure 45(b). Query 8.10: the route as a graph (graph view).

Result: the route has three steps. Each step is justified by shared classes and clubs, for example the first link exists because the two students share the GDG on Campus club and a BAHUM106 class. This is the kind of explanation that is difficult to produce from a relational query.

9. Neo4j in the Application

The queries of Section 8 are also built into the running application. The Campus graph page (sidebar, Campus life) shows the results for the signed-in user. It is the part of Campus Connect that reads from Neo4j.

9.1 The Campus graph page

The top of the page shows the signed-in student's neighbourhood: the twelve people with whom the student has the most in common. Closer and thicker lines mean more shared items. Hovering over a person shows what they share. Beside it, a panel lists the nodes and relationships in the graph.

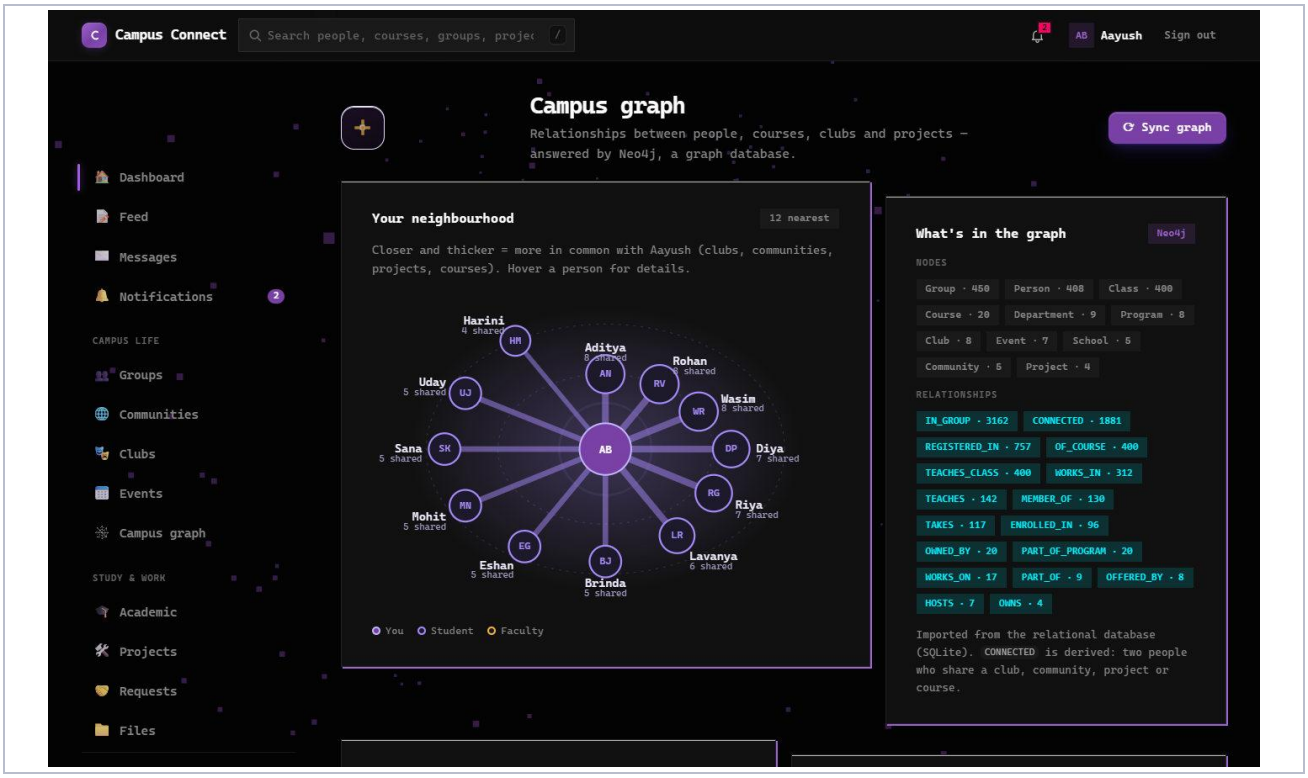


Figure 46. The Campus graph page: neighbourhood diagram and the contents of the graph.

Below the diagram, the page runs the queries of Sections 8.1 to 8.6 and shows each result next to its Cypher (the Cypher panel can be expanded). The shortest-path card has a person selector, so any connected person can be chosen.

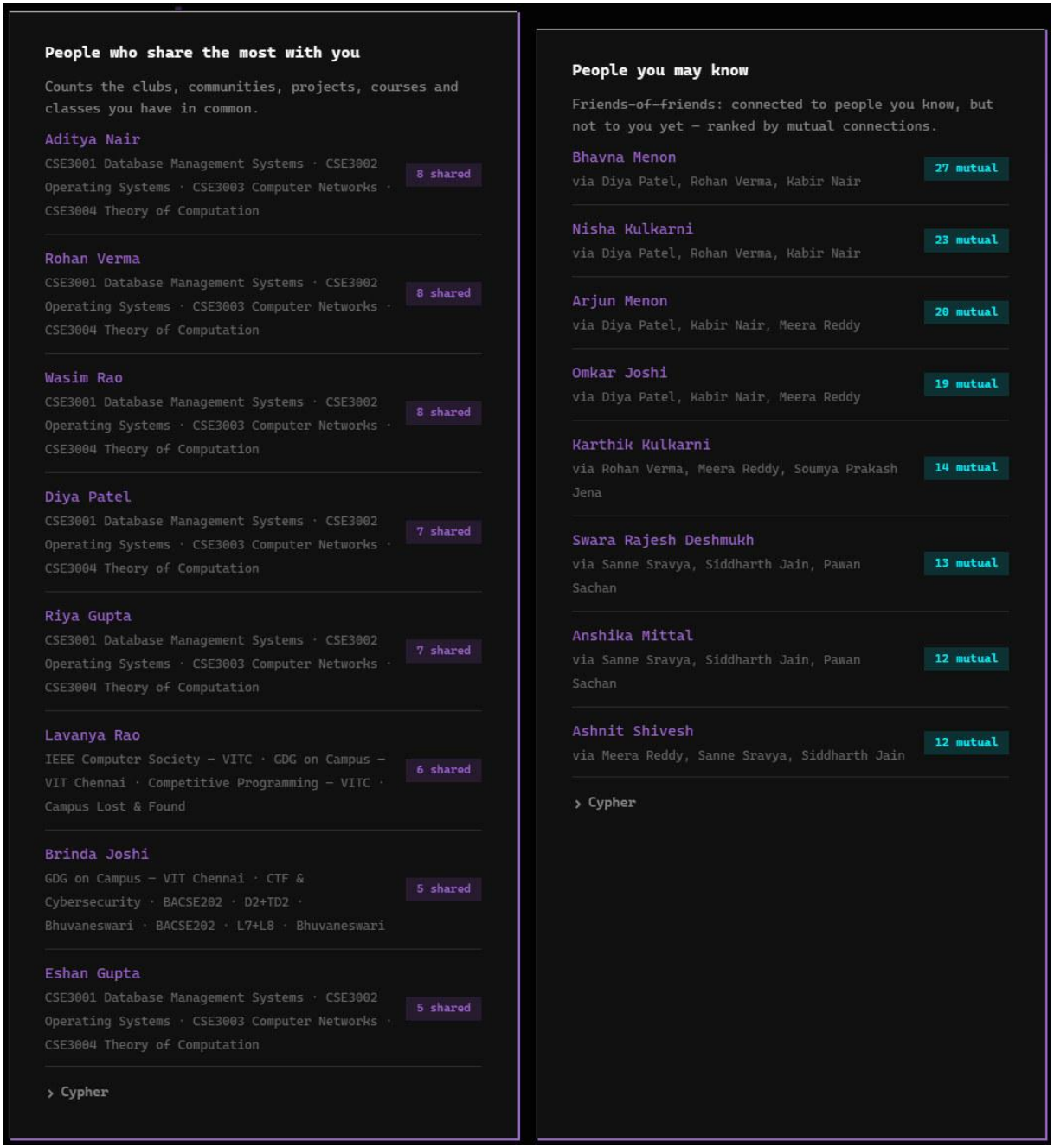


Figure 47. People who share the most with you (left) and people you may know (right), as listed on the page.

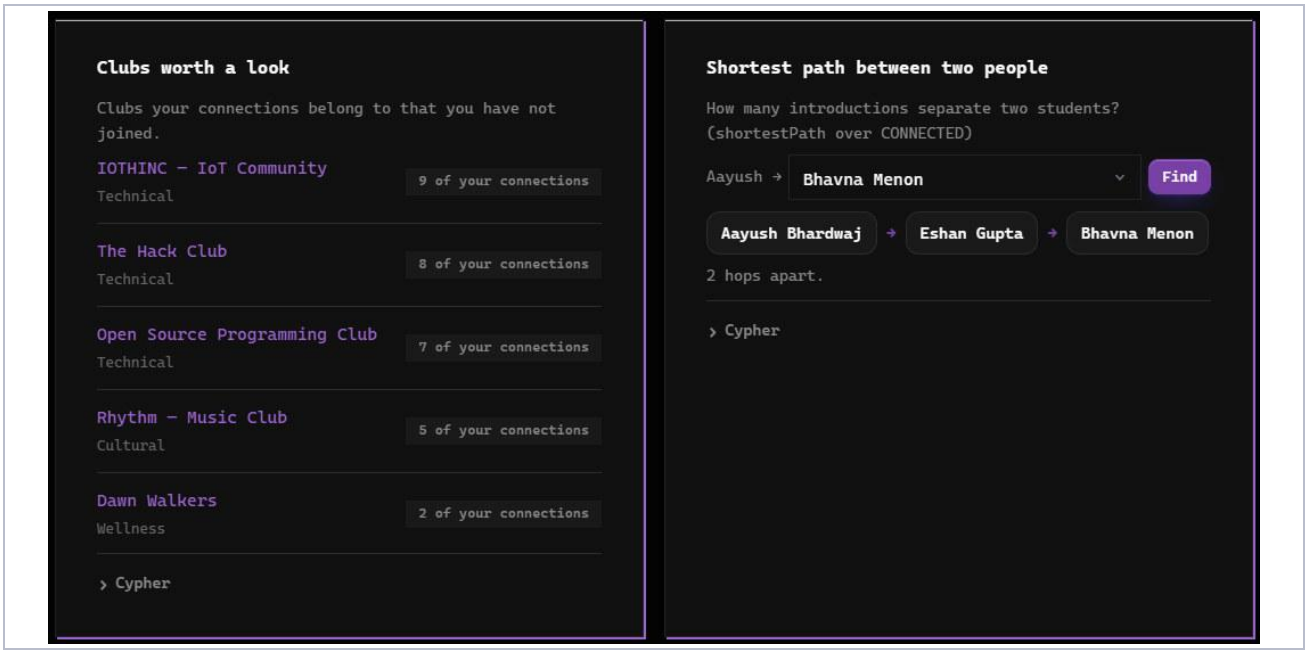


Figure 48. Clubs worth a look (left) and the shortest path between two people, with a person selector (right).

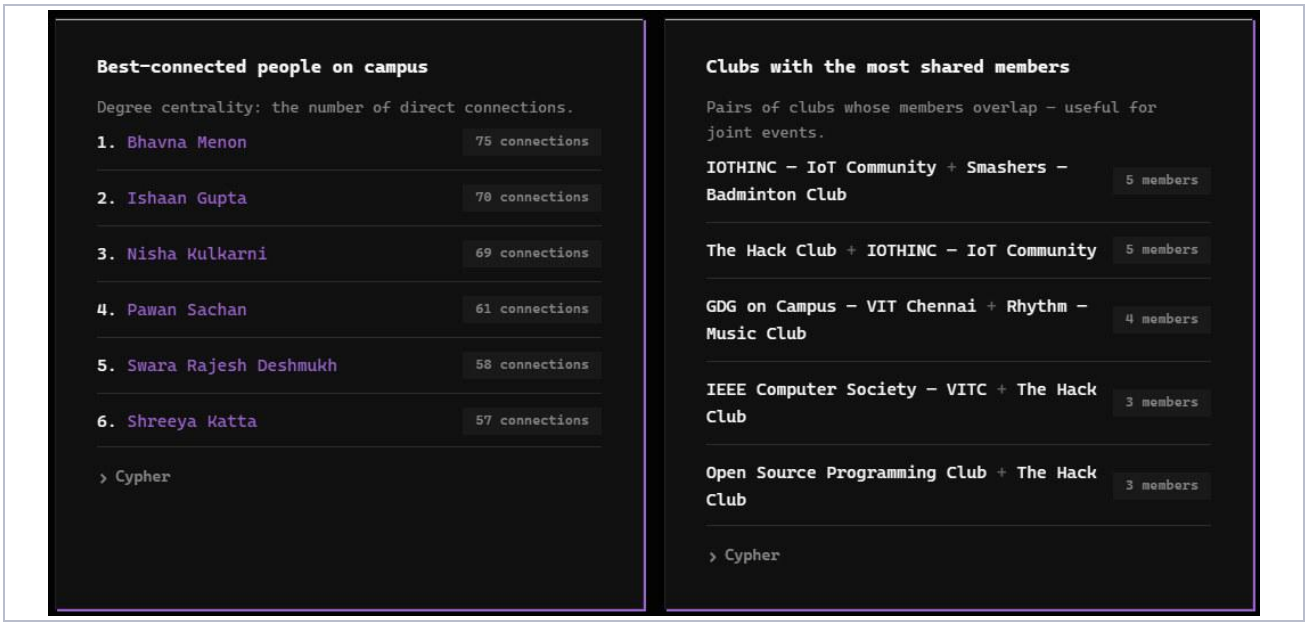


Figure 49. Best-connected people (left) and clubs with the most shared members (right).

9.2 Operations on the Campus graph page

The page is interactive. Three operations let the user work with the graph directly from the application.

Viewing the Cypher behind a result

Every card has a Cypher panel. Expanding it shows the exact query that was sent to Neo4j to produce the list above it, so the result can be traced back to a graph pattern. The figure shows the panel of the people-you-may-know card.

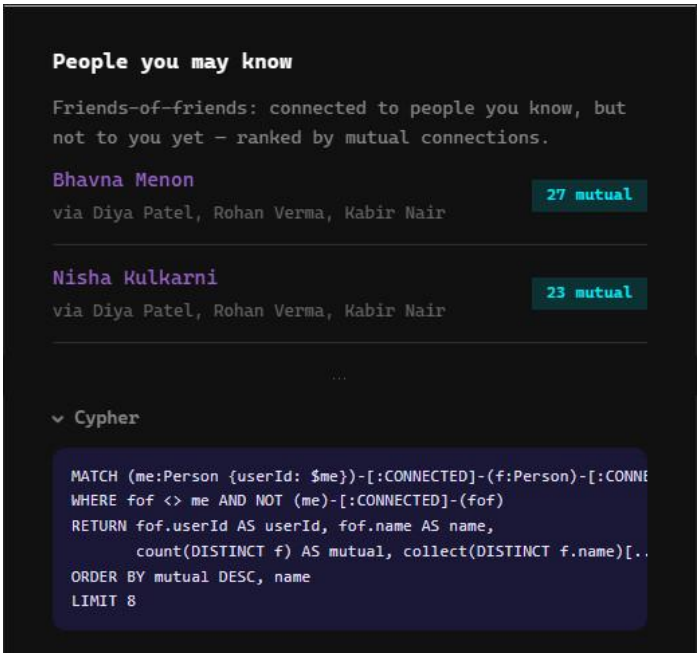


Figure 50. The Cypher panel of the people-you-may-know card, opened to show the two-hop query.

Finding a path to a chosen person

The shortest-path card has a person selector and a Find button. The page asks Neo4j for the shortest chain of CONNECTED relationships between the signed-in student and the selected person, and shows the chain with the number of hops. The first figure shows the default target, who is two hops away. The second shows a different person chosen from the selector, who is three hops away through two intermediate people.

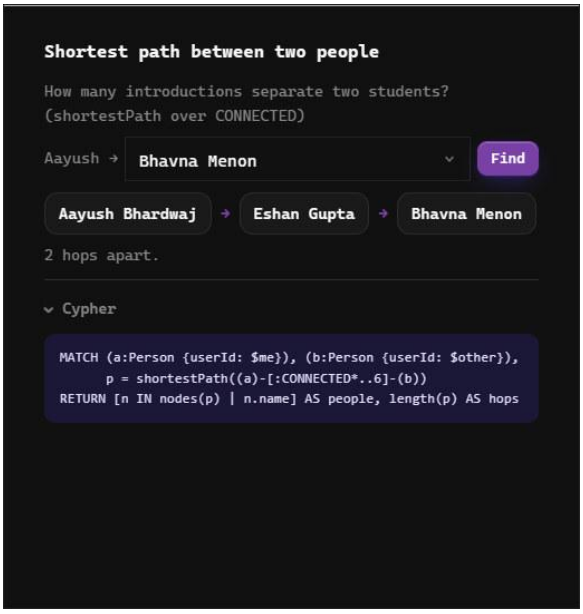


Figure 51. Shortest path to the default target: two hops.

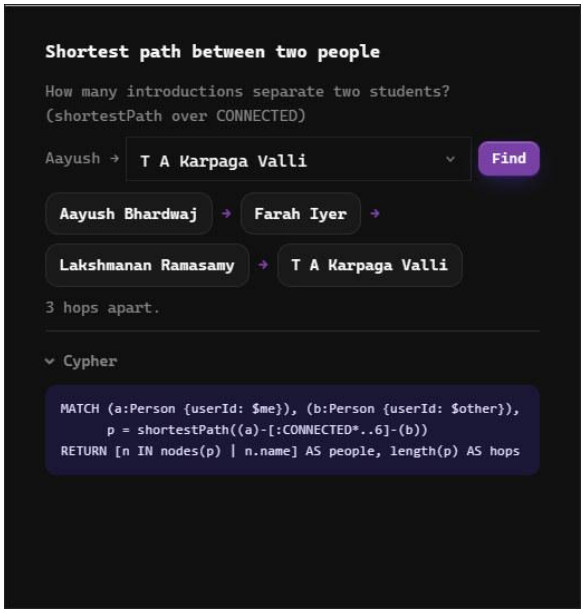


Figure 52. A different person selected and Find pressed: three hops.

Synchronising the graph with the application data

The Sync graph button re-copies the current SQLite data into Neo4j. When it is pressed, the application clears the graph, runs the import again, derives the CONNECTED relationships and reports the result in a banner, here 408 people and 1,881 connections in one second. The graph afterwards holds the same 1,324 nodes and 7,482 relationships, because the application data had not changed.

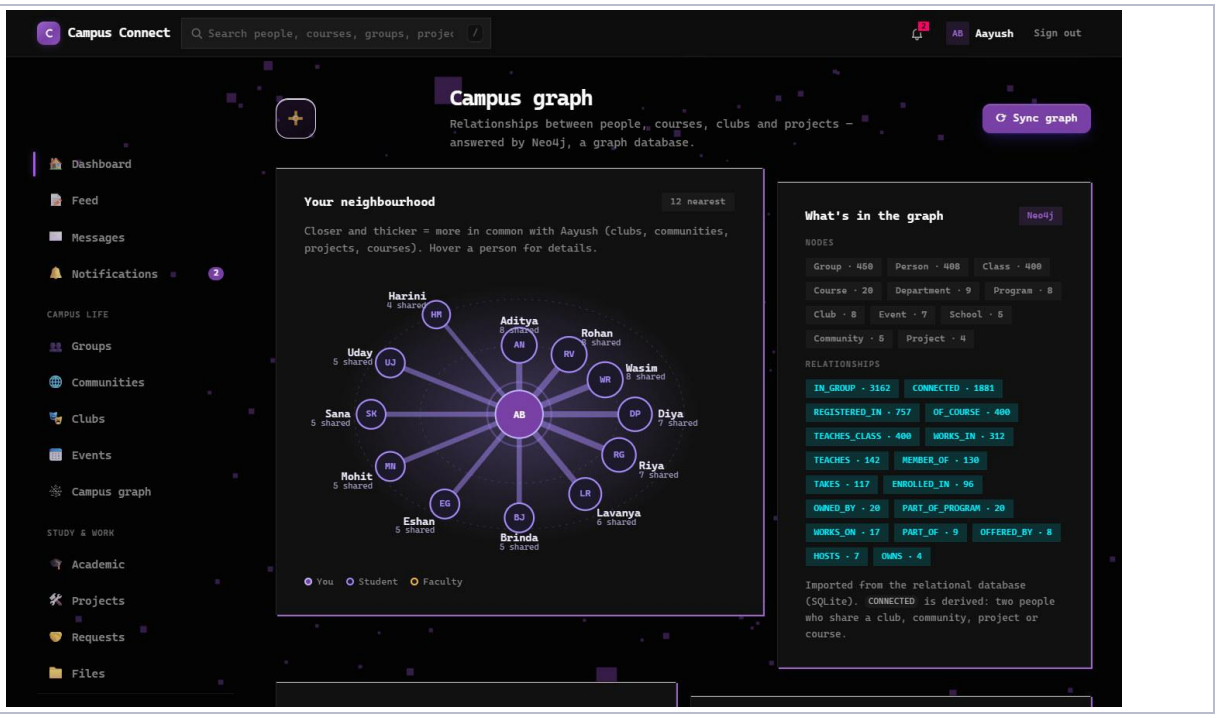


Figure 53(a). Sync graph: the Campus graph page with the Sync graph button.

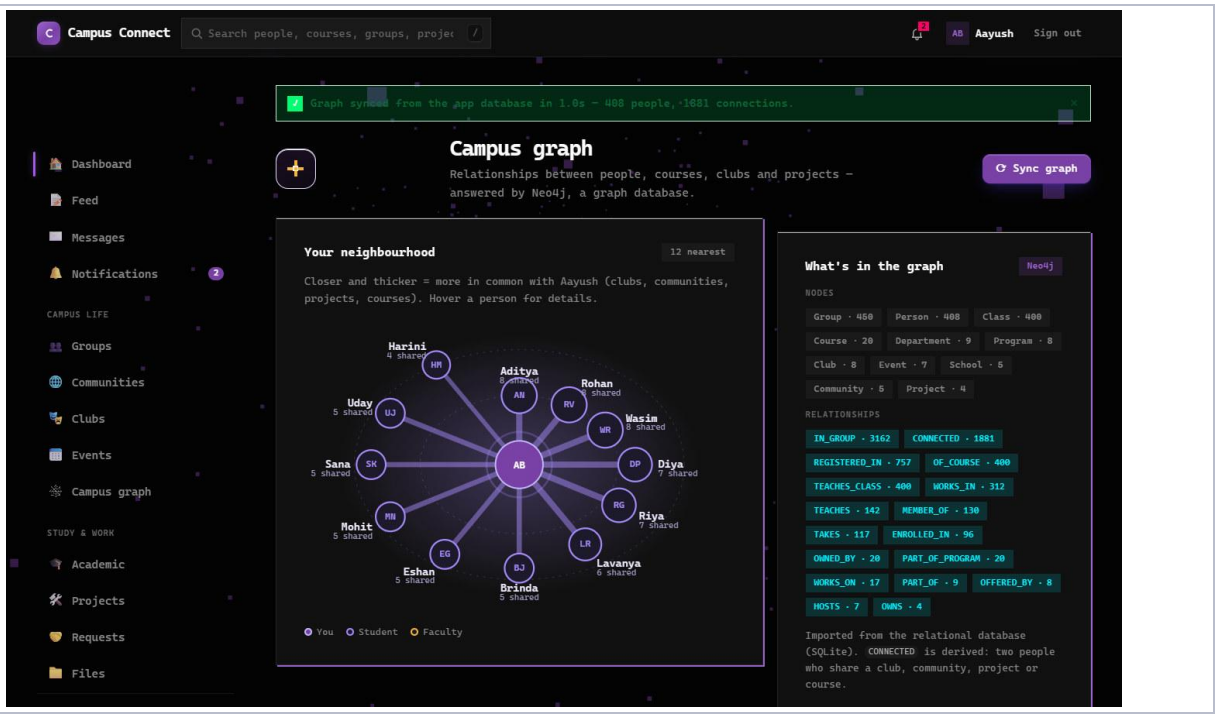


Figure 53(b). Sync graph: after pressing the button: the confirmation banner appears.

9.3 Keeping the graph up to date

The application writes to SQLite; Neo4j is a copy. The Sync graph button on the Campus graph page (a POST to /graph/sync) re-copies the current SQLite data into Neo4j in a few seconds, using the same code as npm run graph:import. If Neo4j is not running, the page shows a message and the rest of the application is unaffected.

9.4 Why two databases

Using more than one database, each for what it does best, is called polyglot persistence. SQLite keeps the transactional, tabular data (accounts, posts, messages, timetables). Neo4j keeps the relationship view of the same data and answers the network questions. The graph can always be rebuilt from SQLite, so there is a single source of truth.

10. Project Demonstration

The demonstration follows the order of the report and takes about five minutes.

1. **Application overview.** Sign in as the demonstration student and show the dashboard, groups, class registration, feed and chat (Section 2.5).
2. **Model.** Show the graph schema and the mapping from SQLite tables to relationships (Section 4).
3. **Implementation and import.** Show the running Neo4j instance, run npm run graph:import and show the counts in Neo4j Desktop (Sections 5 and 6).
4. **Core operations.** Run the create, read, update and delete statements in Neo4j Browser (Section 7).
5. **Queries.** Run the people-you-may-know, shortest-path and reach-this-person queries, in table and graph view (Section 8).
6. **Application.** Open the Campus graph page, change the shortest-path target, and press Sync graph (Section 9).

10.1 How to run the project

Step	Command
Start the graph database	`Start the campus instance in Neo4j Desktop and wait for Running`
Load the graph	`npm run graph:import`
Terminal demonstration of the CRUD operations and queries	`npm run graph:demo`
Start the application	`npm start, then open http://localhost:3000`
Browse the relational data	`npm run sql (read-only SQL prompt) or npm run sqlite:demo`
Rebuild the demonstration data	`npm run seed`

Demonstration login (local development data): aayush@campus.edu with the demonstration password given in the project README.

11. Comparison: Why Neo4j Is Suitable for Campus Connect

11.1 The same question in SQL and in Cypher

Question	Relational (SQLite)	Neo4j (Cypher)
People you may know (friends of friends)	Self-joins over the membership tables; one more join for every extra hop	(me)-[:CONNECTED]-()-[:CONNECTED]-(fof)
How far apart are two students?	Recursive common table expression; awkward to write and slow as data grows	shortestPath((a)-[:CONNECTED*..6]-(b))
Clubs my connections are in that I am not	Four-way join plus NOT EXISTS	One MATCH with WHERE NOT (me)-[:MEMBER_OF]->(c)
Who is best connected?	Group a union of several tables	count of CONNECTED relationships per person
Add a new kind of relationship	New junction table and a schema change	Create the relationship; no schema change
Explain why two people are linked	Rebuild the join and collect the shared rows	The reason is stored on the relationship (via)

11.2 Comparison of the two databases in this project

	SQLite (relational)	Neo4j (graph)
Role in Campus Connect	System of record: accounts, posts, groups, messages, timetables	Relationship questions: connections, paths, recommendations
Data model	37 tables with foreign keys	11 node labels and 17 relationship types
Query language	SQL	Cypher (pattern matching)
Strong at	Integrity, transactions, reports and aggregates	Multi-hop traversal, paths, recommendations
Weaker at	Deep joins and path queries	Bulk reporting and strict multi-row transactions
Server needed	No: a single file used directly by the application	Yes: a database server on port 7687
Keeps data live	Yes: the application writes here	Copy, refreshed with Sync graph

11.3 Why a graph database suits this application

- **The data is naturally a network.** People, classes, clubs, communities, projects and groups are connected in many ways, and the valuable questions are about those connections.
- **Queries are short and readable.** A traversal that needs several joins in SQL is one pattern in Cypher, and a path query needs no recursion.
- **Relationships carry meaning.** Roles and reasons are stored on the relationship, so results can be explained, as in the reach-this-person query.
- **The model is flexible.** A new kind of relationship needs no schema change.
- **It complements, not replaces, SQL.** Transactions and tabular features stay in SQLite, which is why both databases are used.

11.4 Limitations

- The graph is a copy of the relational data and can be out of date until it is synced.
- The CONNECTED relationship follows our own rule (at least two shared items); it is a model of closeness, not a measured social tie.
- A graph database is weaker for bulk reporting and for strict multi-row transactions, so it does not replace the relational database.

12. Conclusion

Adding Neo4j to Campus Connect did not replace the relational design; it gave the application a better tool for one kind of question. The graph model turns the junction tables of the relational schema into relationships, and the 1,324 nodes and 7,482 relationships in the database come from the same SQLite data through a repeatable import. One derived relationship, CONNECTED, makes multi-hop questions simple to ask.

Ten Cypher queries, from friends of friends to student-only introduction paths with explanations, were run against the graph and are also built into the Campus graph page of the application. They show where a graph database is stronger than the relational design, while the comparison also records where SQLite remains the better choice.

What we learned:

- Junction tables map naturally to relationships, and their extra columns become relationship properties.
- Friends-of-friends, shortest-path and recommendation queries are a few lines of Cypher, where SQL needs several joins or a recursive query.
- The rule that derives relationships changes the graph a great deal; a poor threshold makes every path one hop long.
- Two databases need a clear rule for which one is the truth. In Campus Connect it is SQLite, and Neo4j is refreshed from it.

Possible future work

- Keep the graph live by writing to Neo4j when memberships change, instead of syncing on demand.
- Use graph algorithms such as community detection to suggest study groups automatically.

References

- Neo4j Cypher Manual: neo4j.com/docs/cypher-manual
- Neo4j JavaScript Driver Manual: neo4j.com/docs/javascript-manual
- SQLite documentation: sqlite.org/docs.html
- Project source, README and demonstration scripts (DA3.md and DA3_DEMO_SCRIPT.txt): https://github.com/aayushbhardwaj2506/CAMPUS-CONNECT/tree/da3-neo4j-graph